Academic Record Management System With Mini DBMS

Submitted By

Student Name	Student ID
Md Ashikur Alom Sarkar	253-15-204
Aditya Nag Saptam	253-15-558
Athoi Mondal Katha	253-15-439

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course
CSE114: Programming and Problem Solving Laboratory in the Computer Science and Engineering Department

DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

April 22, 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Md. Ashraful Islam Talukder**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Md. Ashraful Islam Talukder
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted by

Student Name: Md Ashikur Alom Sarkar
ID: 253-15-204
Dept. of CSE, DIU

Student Name: Aditya Nag Saptam
ID: 253-15-558
Dept. of CSE, DIU

Student Name: Athoi Mandal Katha
ID: 253-15-439
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course has course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Learn a basic programming language (C/C++) to imitate and manipulate various problem solving techniques.
CO2	Demonstrate skills to analyze competitive programming problems and identify to choose appropriate language constructs and data structures for designing, building and testing realistic, complex applications.
CO3	Work collaboratively in teams to design, develop, and implement a comprehensive software project, demonstrating effective communication, coordination, and integration of team efforts to meet project goals.
CO4	Communicate complex engineering concepts effectively through reports, presentations, and instructions to both technical and non-technical audiences.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2, P1, P2, A1, A2	K1, K2, K3, K4	EP1, EP2, EP3
CO2	PO2	C3, C4, P2, P3, A2, A3	K1, K2, K3, K4	EP1, EP2, EP3
CO3	PO9	C3, C4, C5, P3, P4, A3, A4		
CO4	PO10	C6, P2, A4		

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Chapter 1

Introduction

This chapter introduces the Academic Record Management System project, detailing the background of the problem and the motivation behind developing a custom terminal-based DBMS in C. It also outlines the project's core objectives, feasibility, gap analysis, and the expected outcomes for efficient student data management.

1.1 Introduction

Educational institutions process massive amounts of student data daily, ranging from personal demographics to detailed academic performance metrics. Traditionally, managing this critical information relied heavily on manual record-keeping using physical paper ledgers or fragmented, disorganized spreadsheets. While functional for very small cohorts, these traditional methods are highly prone to human error, data loss, and severe operational inefficiencies. As the student population grows, administrators face a significant bottleneck: manually retrieving specific student records, accurately calculating Cumulative Grade Point Averages (CGPAs), ranking students based on merit across different departments, and generating comprehensive report cards becomes overwhelmingly tedious and time-consuming. Furthermore, the lack of a centralized, secure mechanism exposes sensitive academic records to unauthorized access or tampering. Without a structured relational data management approach, educational bodies constantly struggle with data redundancy and inconsistency. To resolve these critical issues, it is highly recommended to implement an automated, computationally driven solution to streamline administrative workflows. This project proposes the development of an "Academic Record Management System with Mini DBMS" utilizing the C programming language. This system completely digitizes the academic tracking process by implementing a custom terminal-based database engine that securely and relationally links personal details with academic achievements using the student ID as a unique primary key. The proposed solution features automated CGPA calculation, an integrated sorting and ranking algorithm, advanced multi-parameter search capabilities, and robust administrative security via encrypted login credentials. By leveraging dynamic memory allocation and persistent file-based storage, this system ensures scalable, error-free, and highly reliable management of student records.

1.2 Motivation

The computational motivation for this project is to replace slow, error-prone manual record-keeping with a fast, structured, and secure digital database. Handling large datasets computationally ensures quick retrieval, automated sorting, and persistent storage. Solving this problem personally benefits us by deeply reinforcing our understanding of core C programming concepts, particularly dynamic memory allocation, relational data mapping, and file I/O operations. It is highly recommended that educational administrators adopt this computational approach over paper ledgers to guarantee data integrity, eliminate redundancy, and significantly reduce the administrative workload associated with calculating grades and generating report cards.

1.3 Objectives

The primary objectives of developing this Academic Record Management System are as follows:

1. To develop a terminal-based relational Mini DBMS in C using primary key mapping.
2. To implement scalable CRUD (Create, Read, Update, Delete) operations using dynamic memory allocation.
3. To automate academic calculations, including CGPA, letter grades, and merit-based ranking via a custom Quick Sort.
4. To ensure data persistence and prevent data loss through text-based file I/O operations.
5. To integrate secure administrative access utilizing hidden inputs and XOR password encryption.
6. To streamline student data retrieval with multi-parameter search, departmental filtering, and automated report card generation.

1.4 Feasibility Study

Existing Student Information Systems are often resource-intensive, relying on complex web frameworks and heavy SQL databases. However, a review of similar console-based applications demonstrates that lightweight, offline systems are highly feasible and effective for low-resource environments. Methodologically, foundational literature [1] emphasizes the importance of efficient algorithm design for sorting and retrieving data within such systems. This C-based project proves highly feasible by successfully implementing core RDBMS concepts—like relational structs and dynamic memory—using only standard libraries. It is recommended to utilize this lightweight architecture as a foundational, easily deployable model for institutions requiring a fast, resource-efficient database solution.

1.5 Gap Analysis

1. The absence of automated computational tools in traditional methods for instantly calculating CGPAs and determining academic letter grades.
2. The lack of structured relational mapping in basic flat-file records to securely link a student's personal details with their academic performance.
3. The inefficiency of manual sorting methods for determining student merit ranks, highlighting the need for integrated algorithmic sorting.
4. The inadequate security measures in standard offline ledgers, exposing sensitive student data and administrative access to unauthorized modifications.
5. The heavy resource dependence of complex web-based databases, creating a demand for a localized, lightweight, and fast terminal-based management system.

1.6 Project Outcome

1. A fully functional, lightweight terminal-based application capable of performing scalable CRUD operations on student records using dynamic memory allocation.
2. Automated and error-free calculation of academic metrics, including instant CGPA generation and merit-based ranking using algorithmic sorting.
3. Secure and persistent data storage achieved through robust text-based file I/O handling and encrypted administrator access.
4. Streamlined administrative workflows through integrated multi-parameter search functions and instant individual report card generation.
5. A deployable mini-DBMS model that practically demonstrates the integration of core C programming concepts, relational data structures, and algorithmic efficiency.

Chapter 2

Architecture

This chapter presents the architectural framework and operational flow of the Academic Record Management System. It exclusively focuses on the design specifications of the C-based Mini DBMS, featuring a comprehensive flowchart that illustrates user interactions and data processing pathways.

2.1 Design Specification

main Function

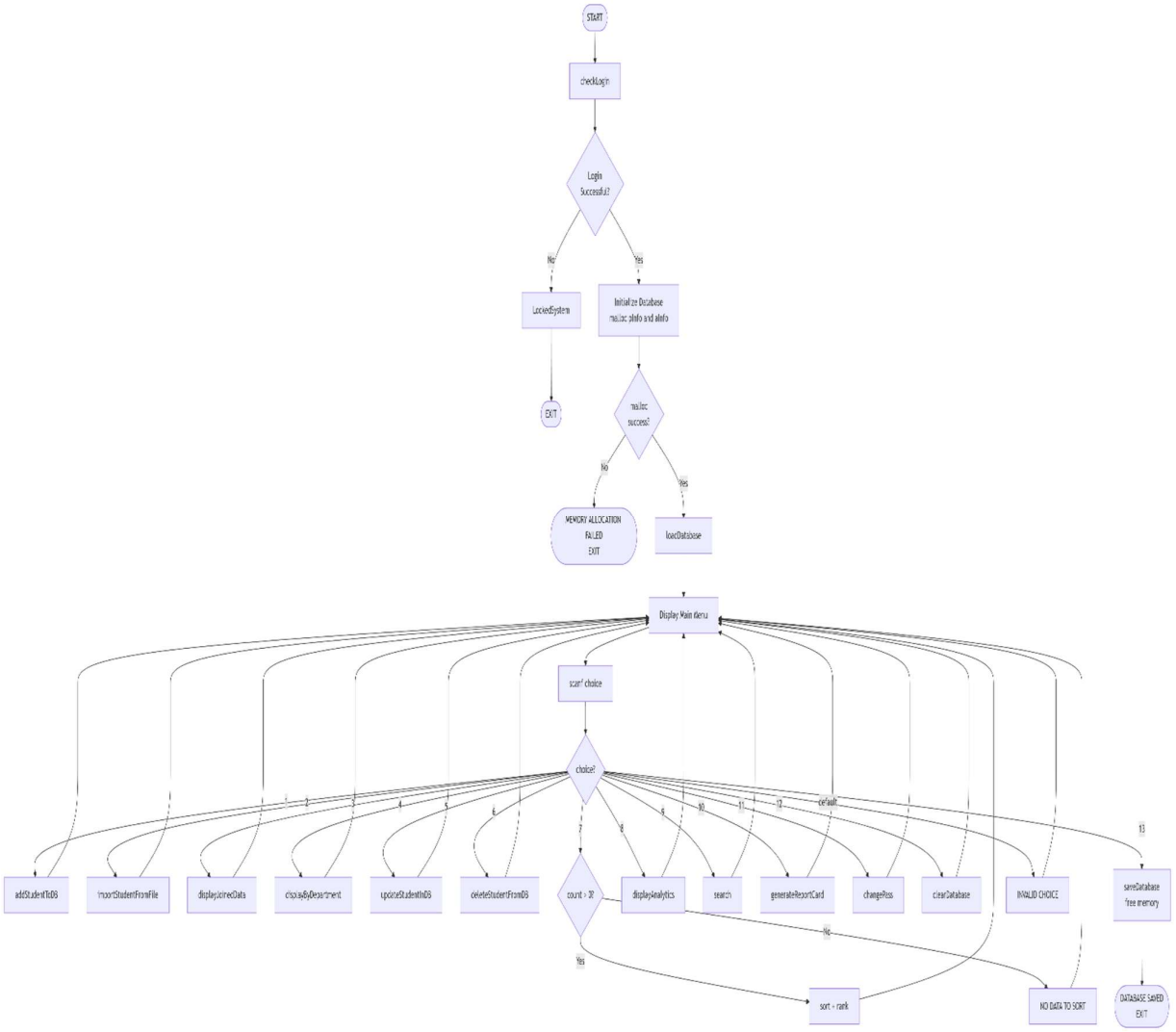


Fig: 2.1 (Main Function)

checkLogin Function

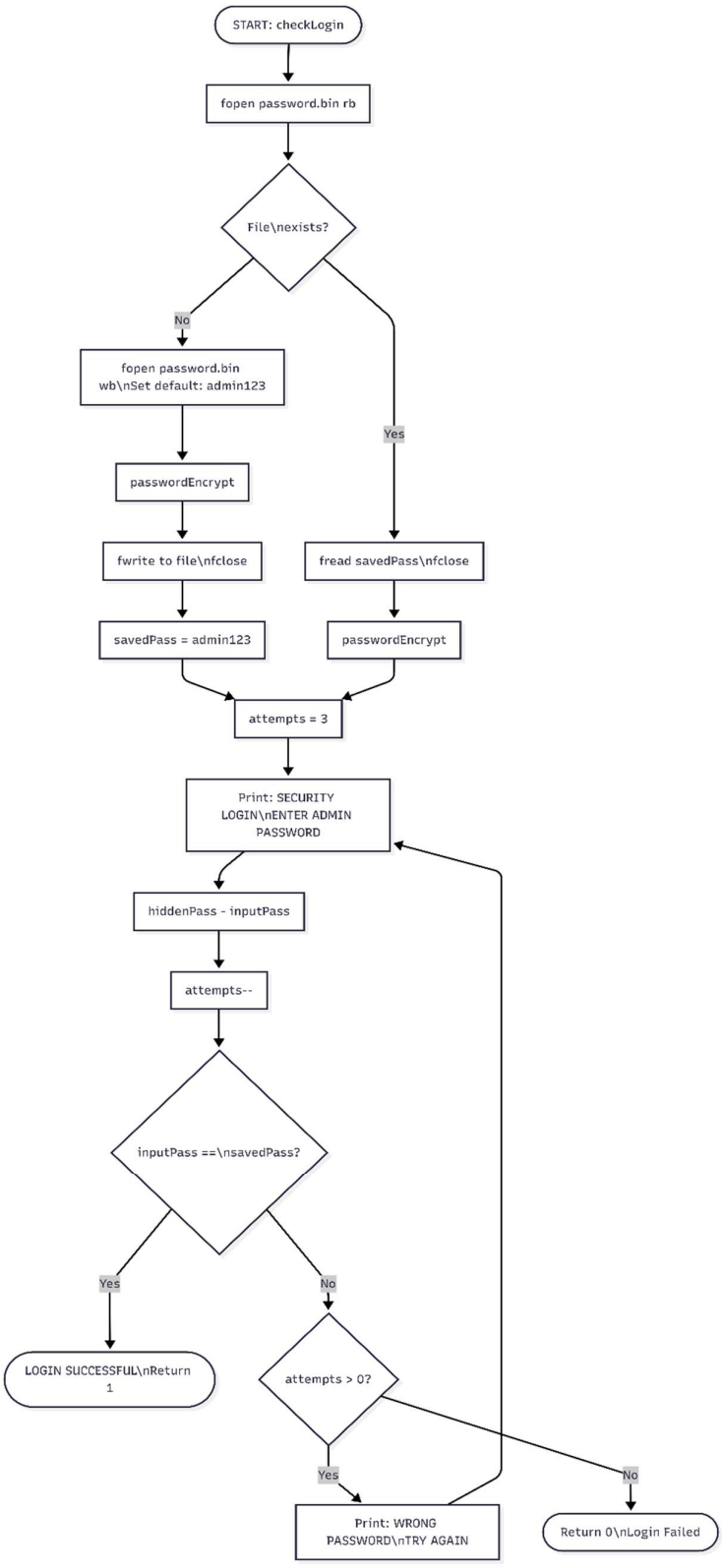


Fig: 2.2 (checkLogin Function)

hiddenPass Function

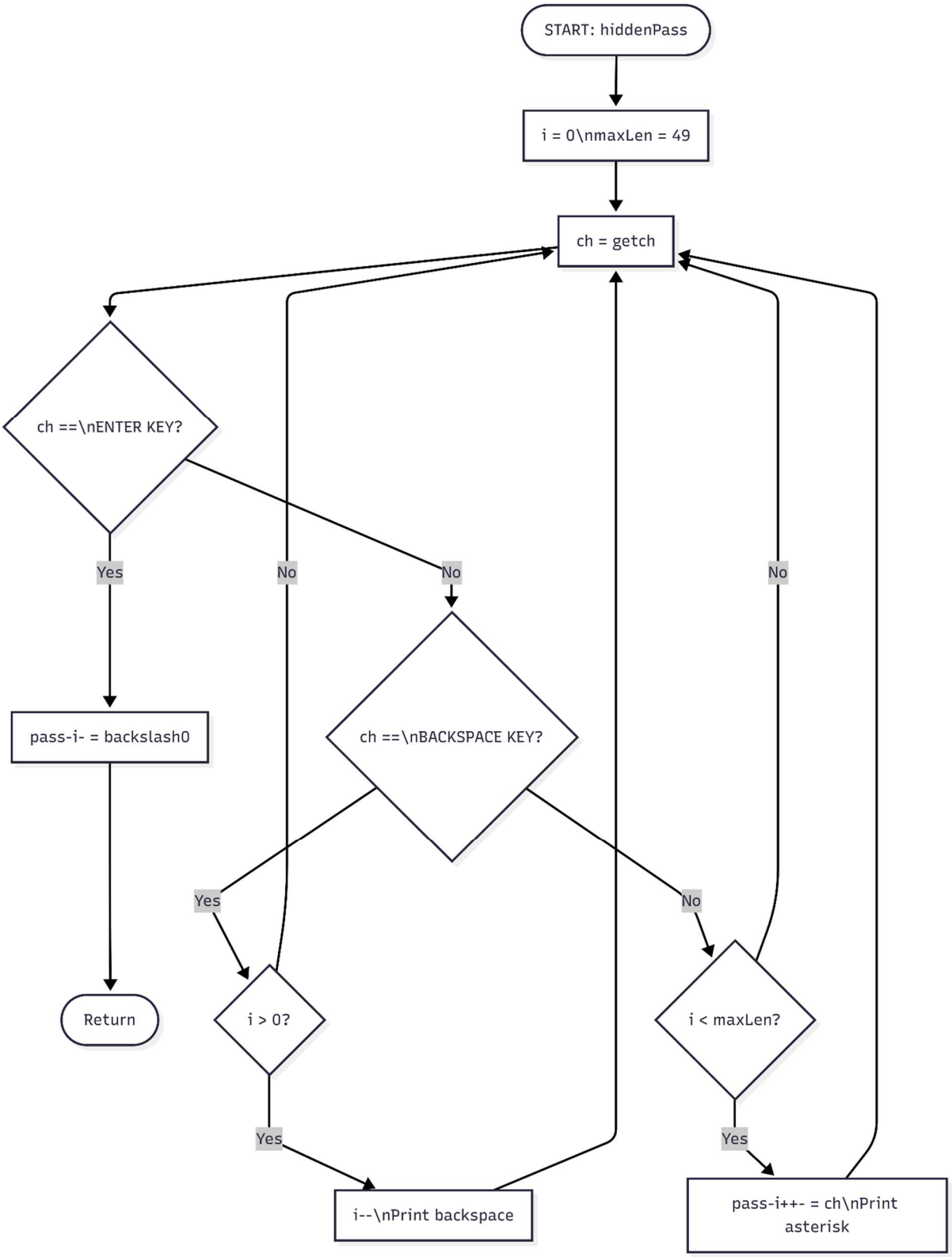


Fig: 2.3 (hiddenPass Function)

passwordEncrypt Function

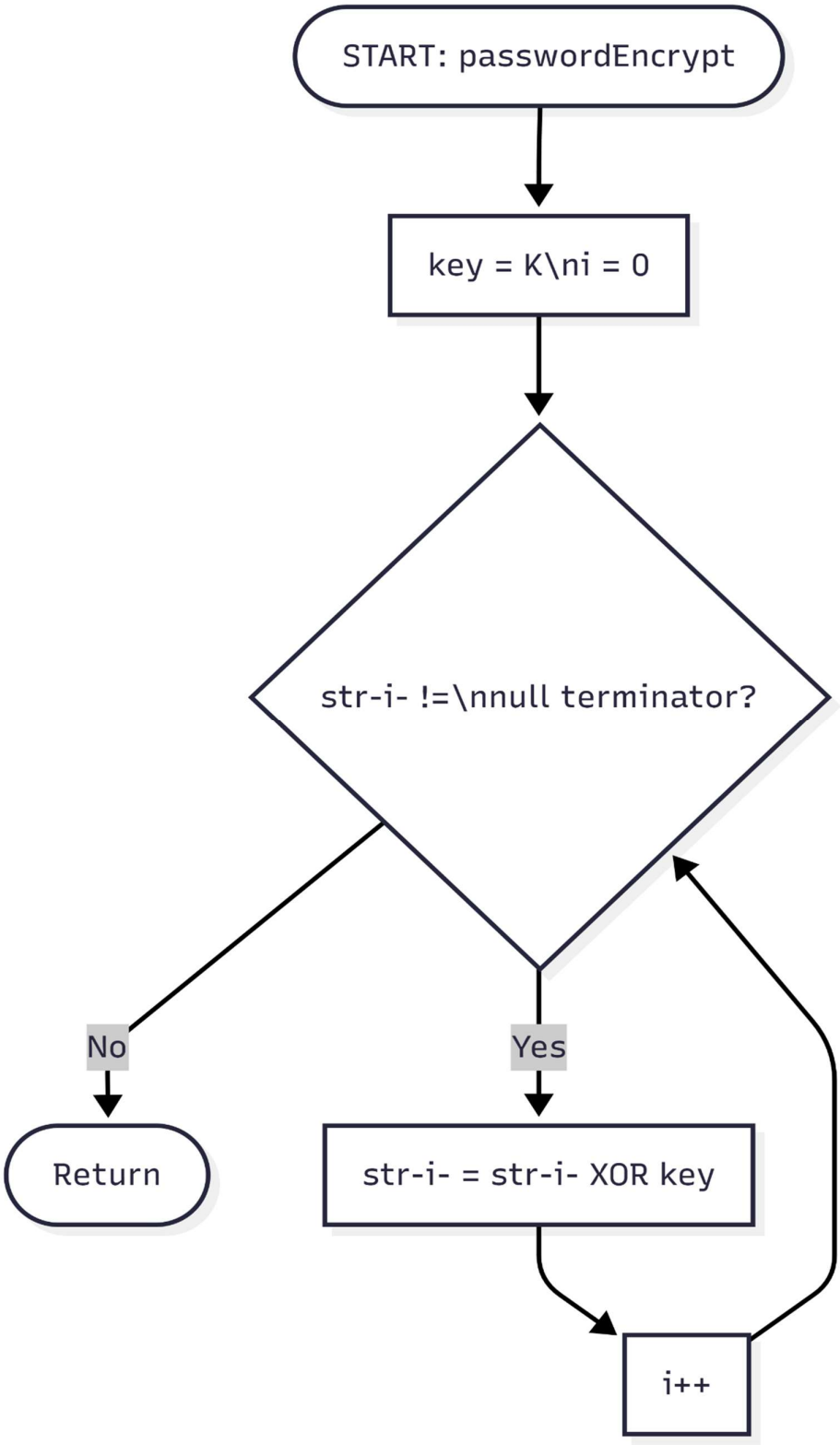


Fig: 2.4 (passwordEncrypt Function)

lockedSystem Function

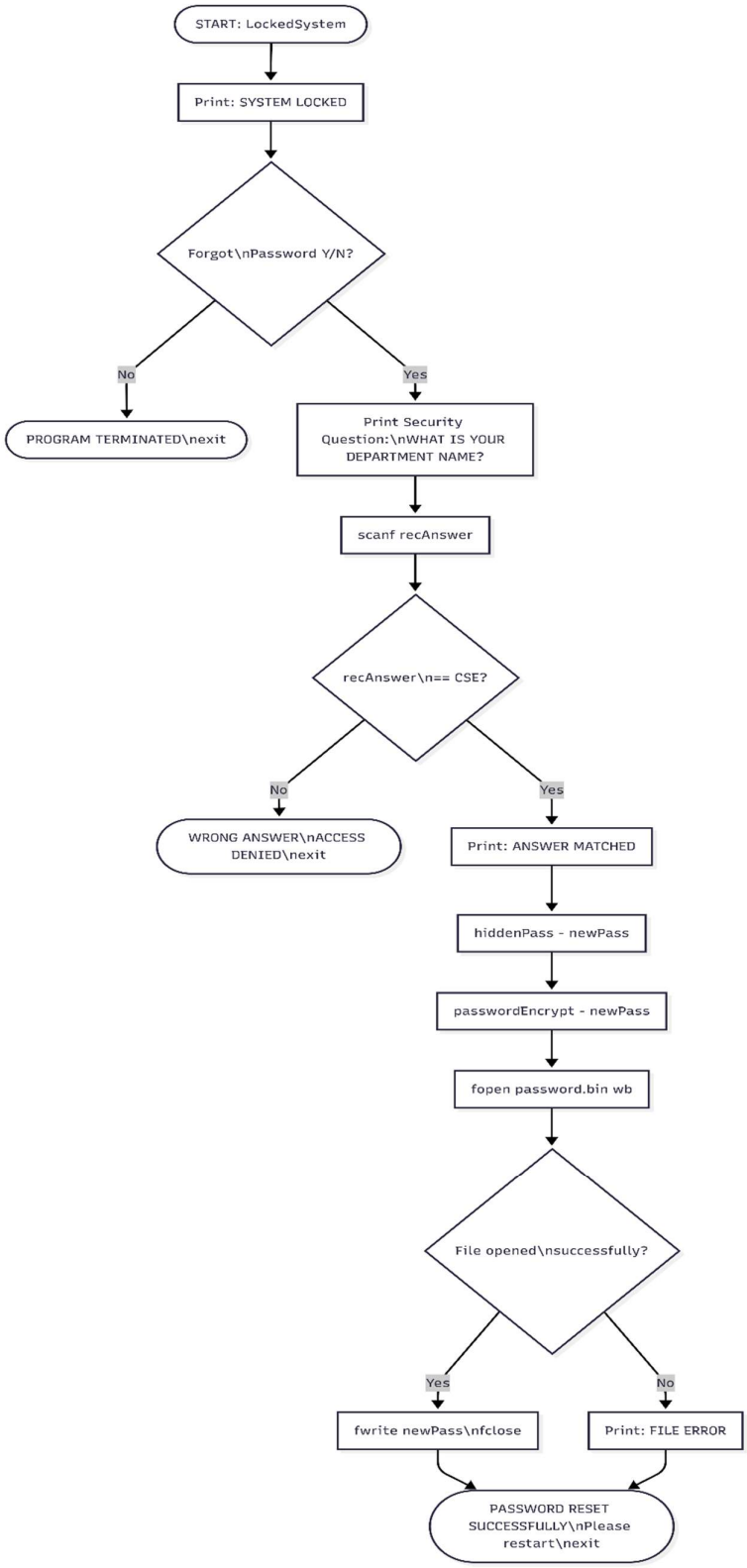


Fig: 2.5 (lockedSystem Function)

changePass Function

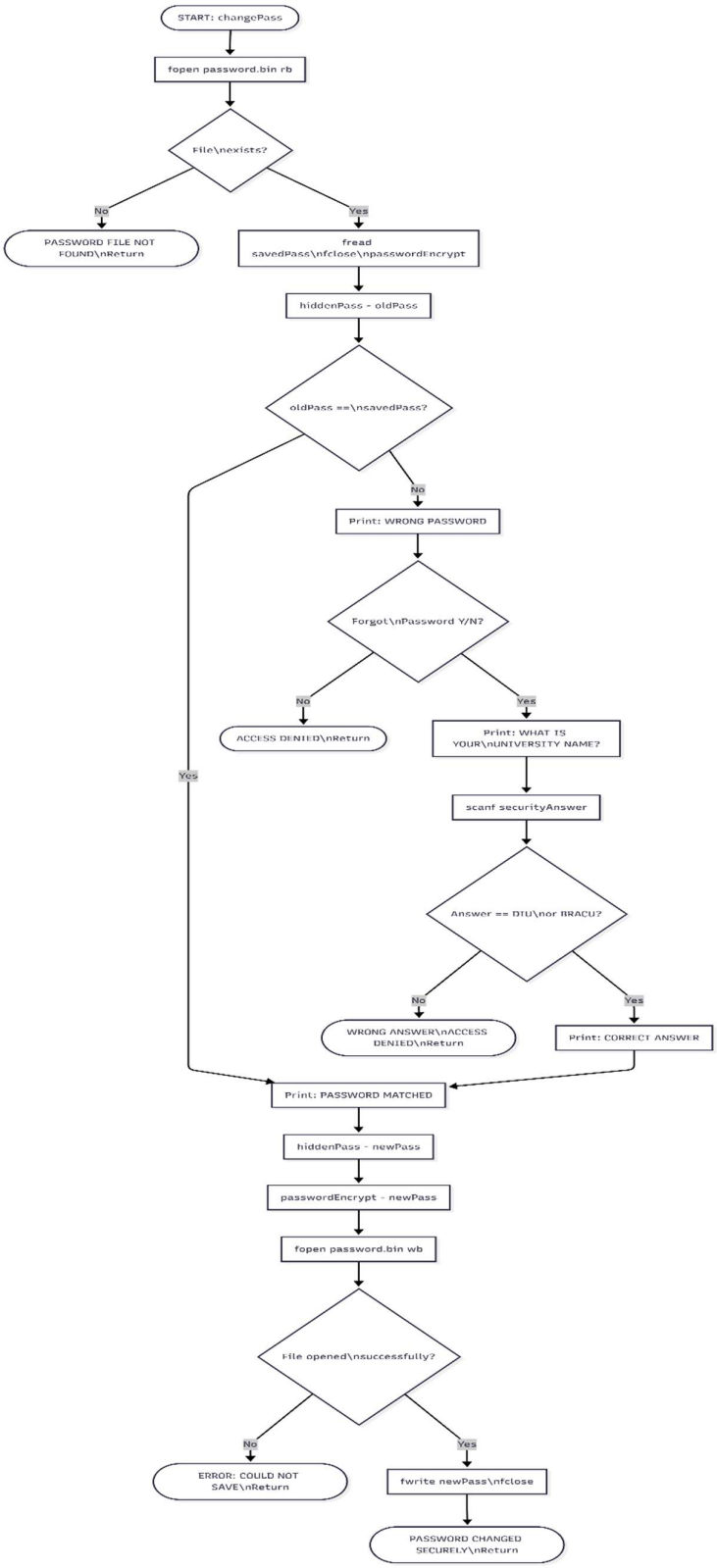


Fig: 2.6 (changePass Function)

addStudentToDB Function

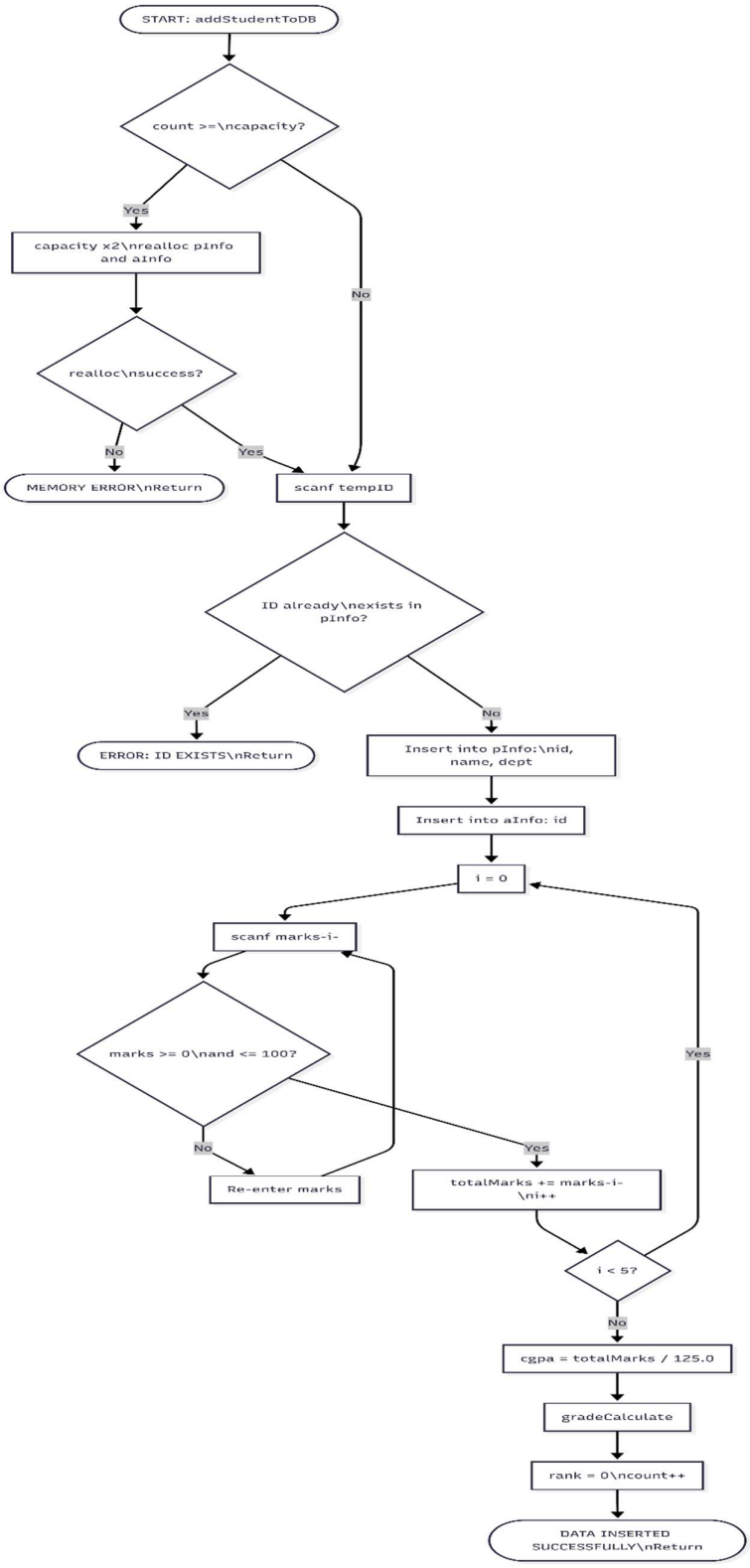


Fig: 2.7 (addStudentToDB Function)

importStudentFromFile Function

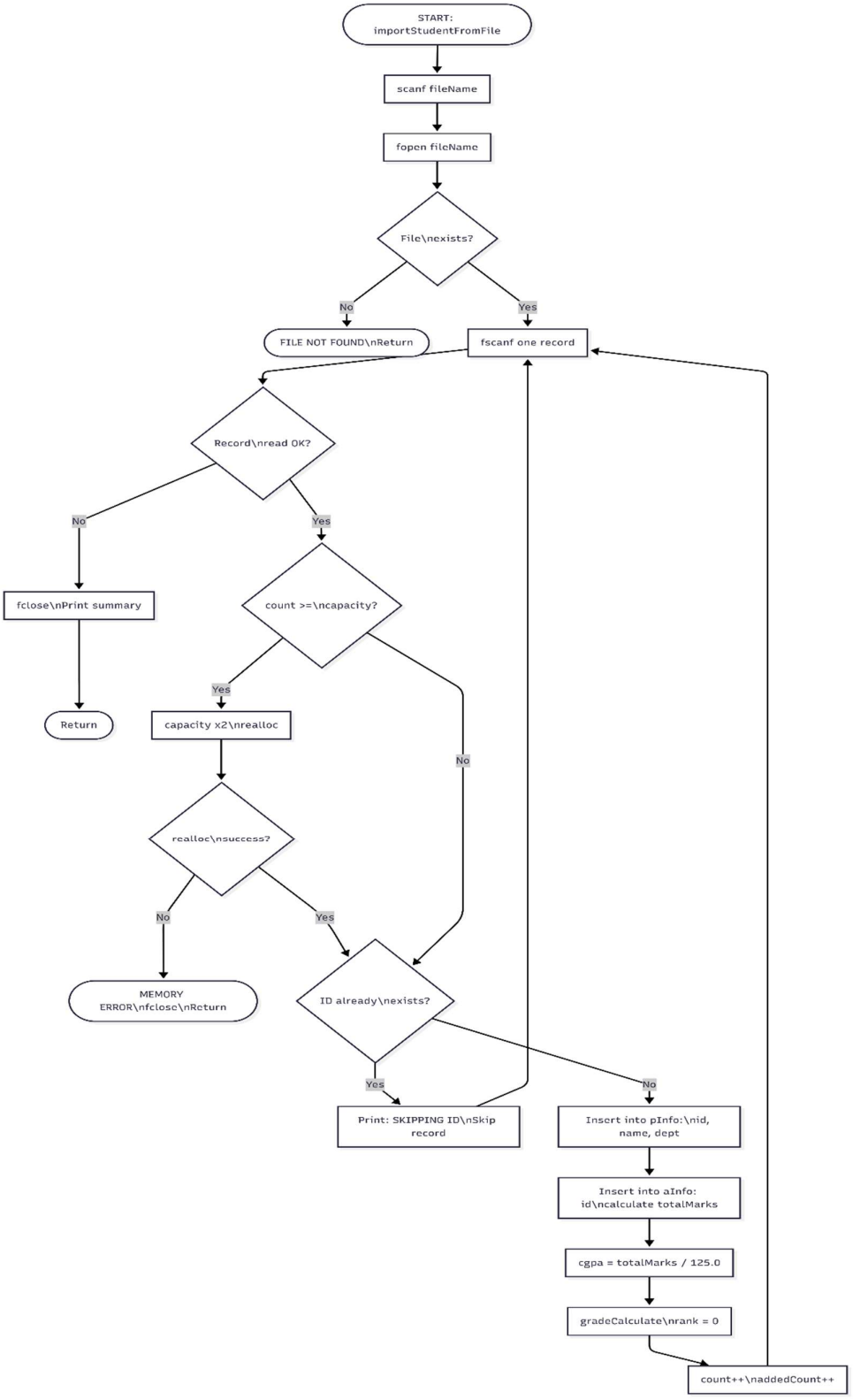


Fig: 2.8 (importStudentFromFile Function)

gradeCalculate Function

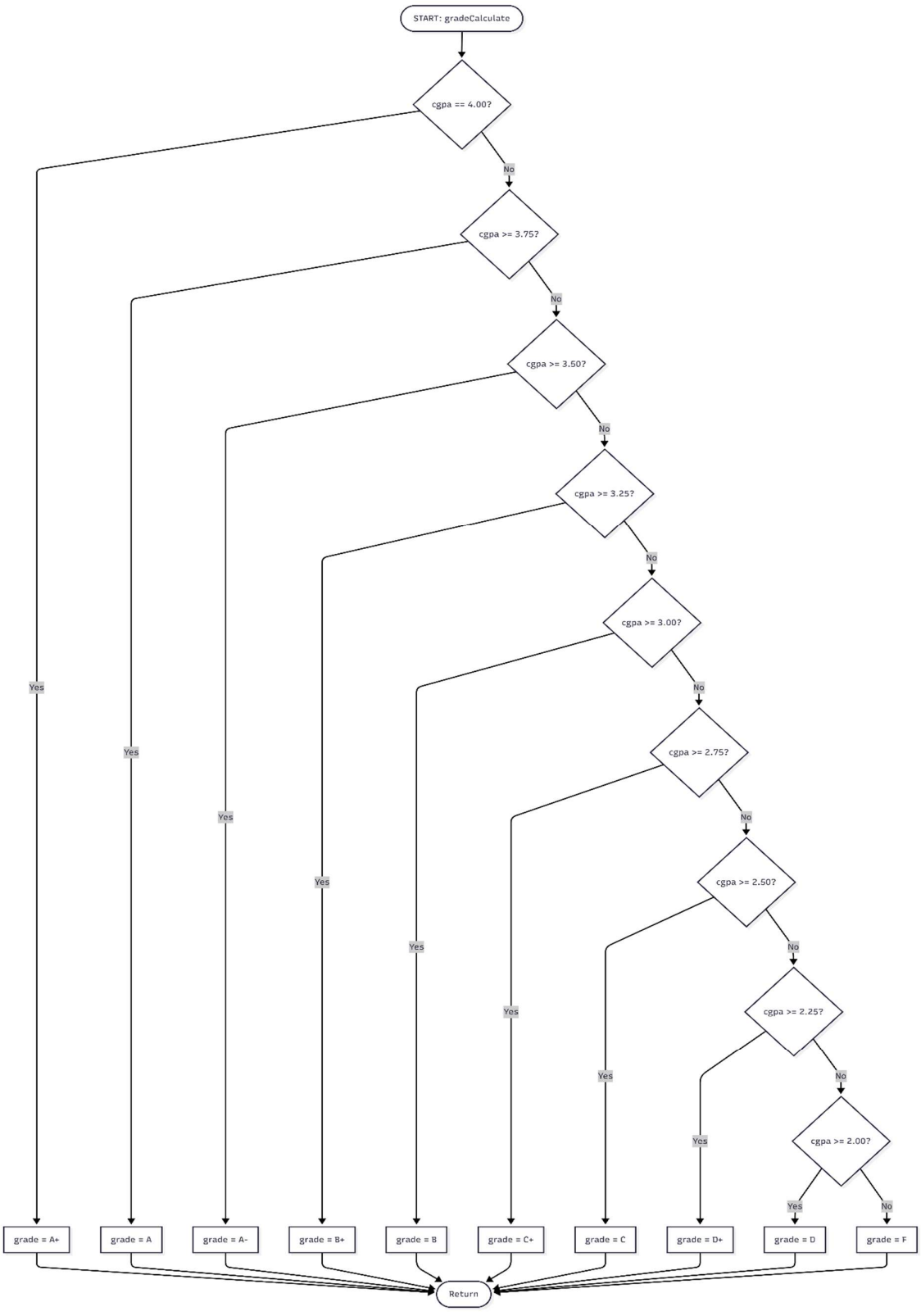


Fig: 2.9 (gradeCalculate Function)

displayJoinedData Function

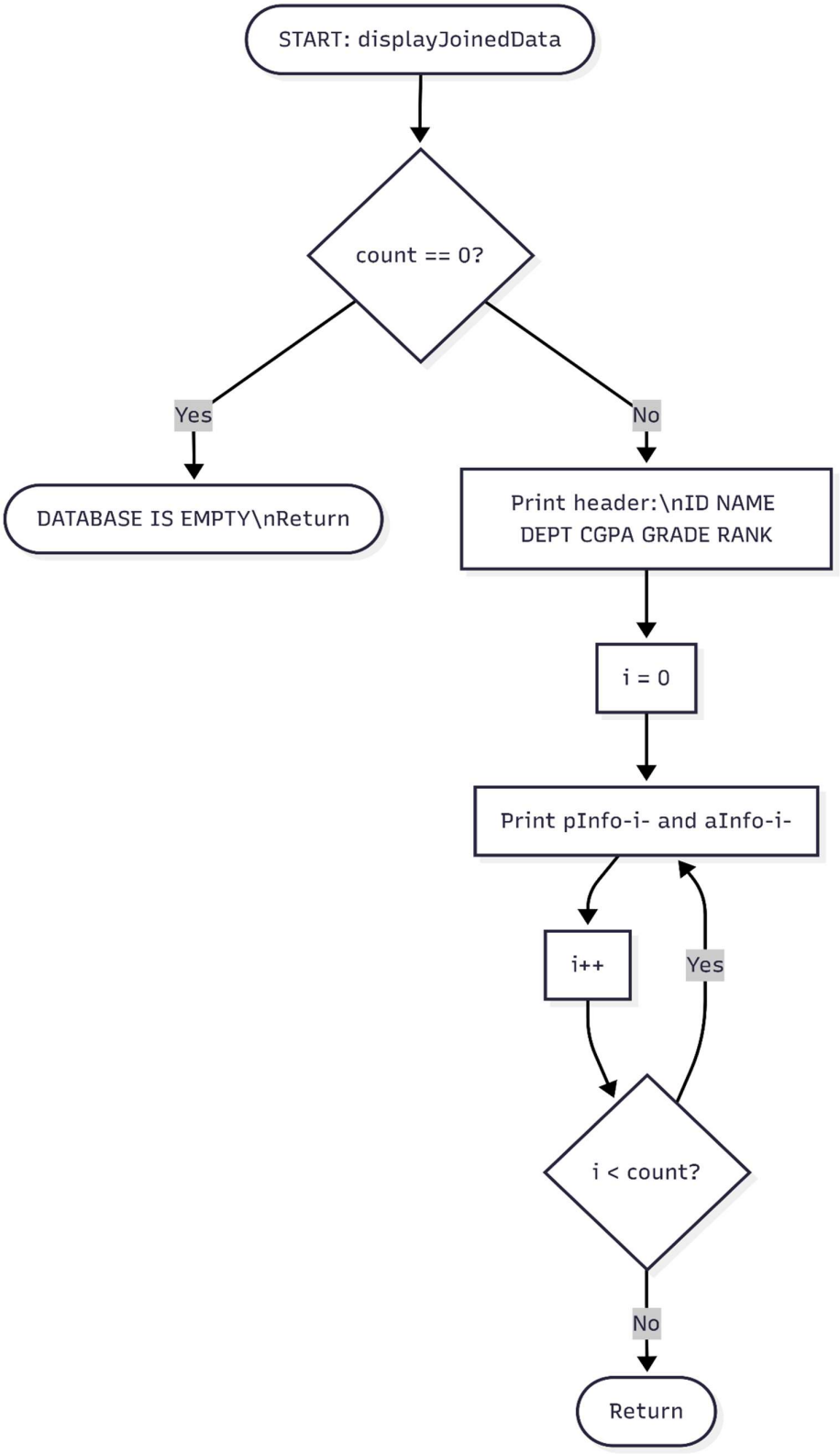


Fig: 2.10 (displayJoinedData Function)

displayByDepartment Function

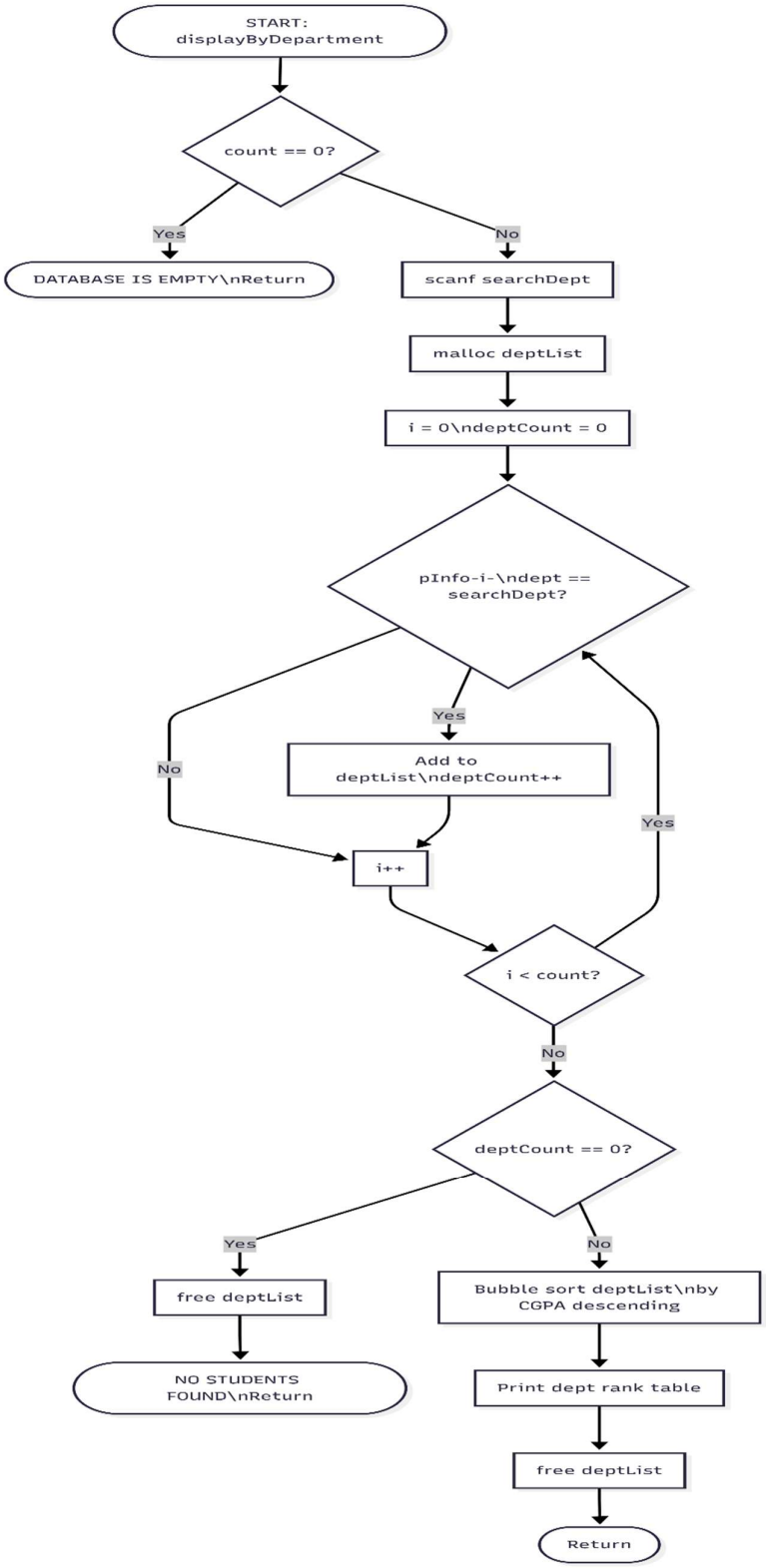


Fig: 2.11 (displayByDepartment Function)

search Function

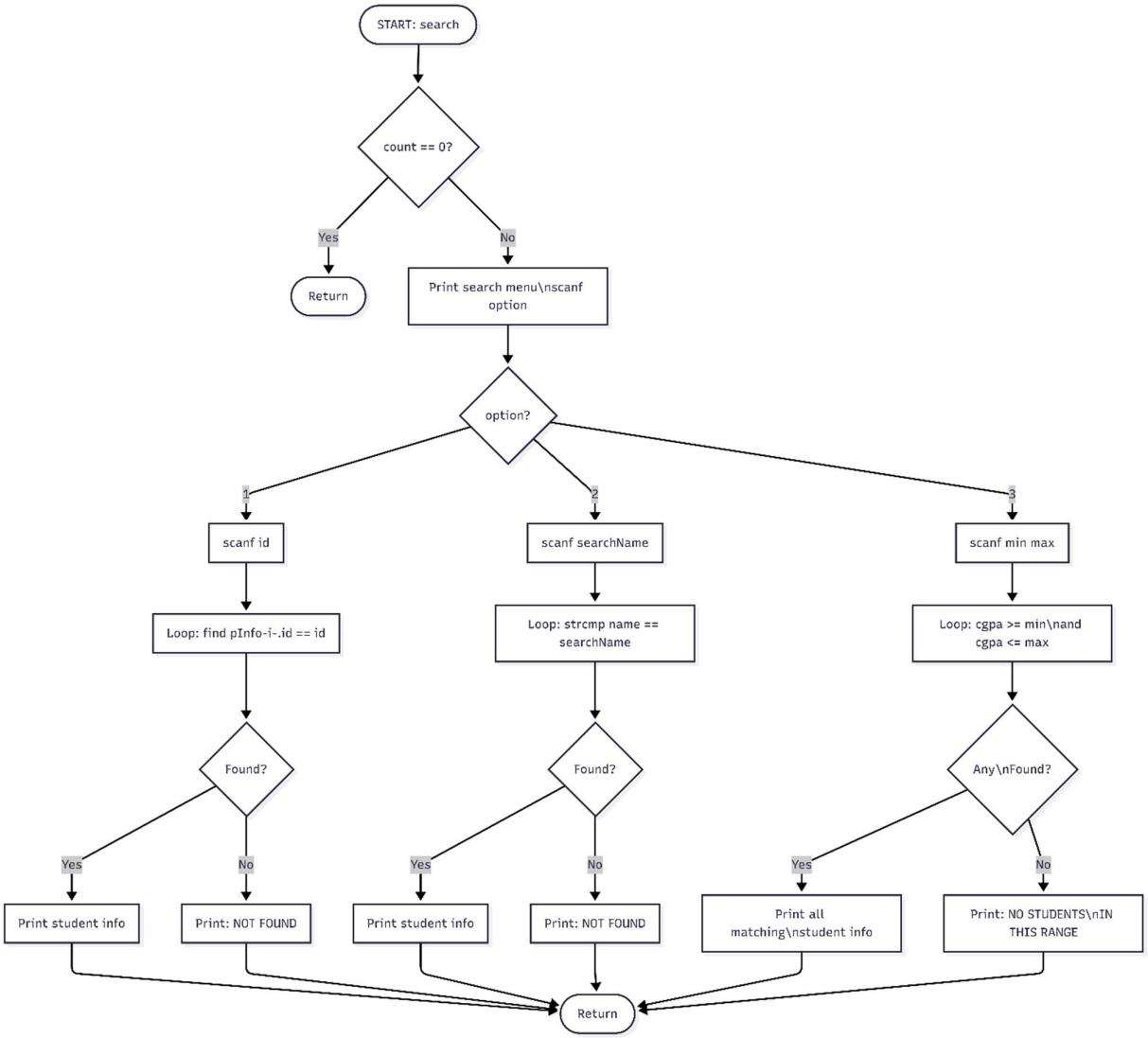


Fig: 2.12 (search Function)

generateReportCard Function

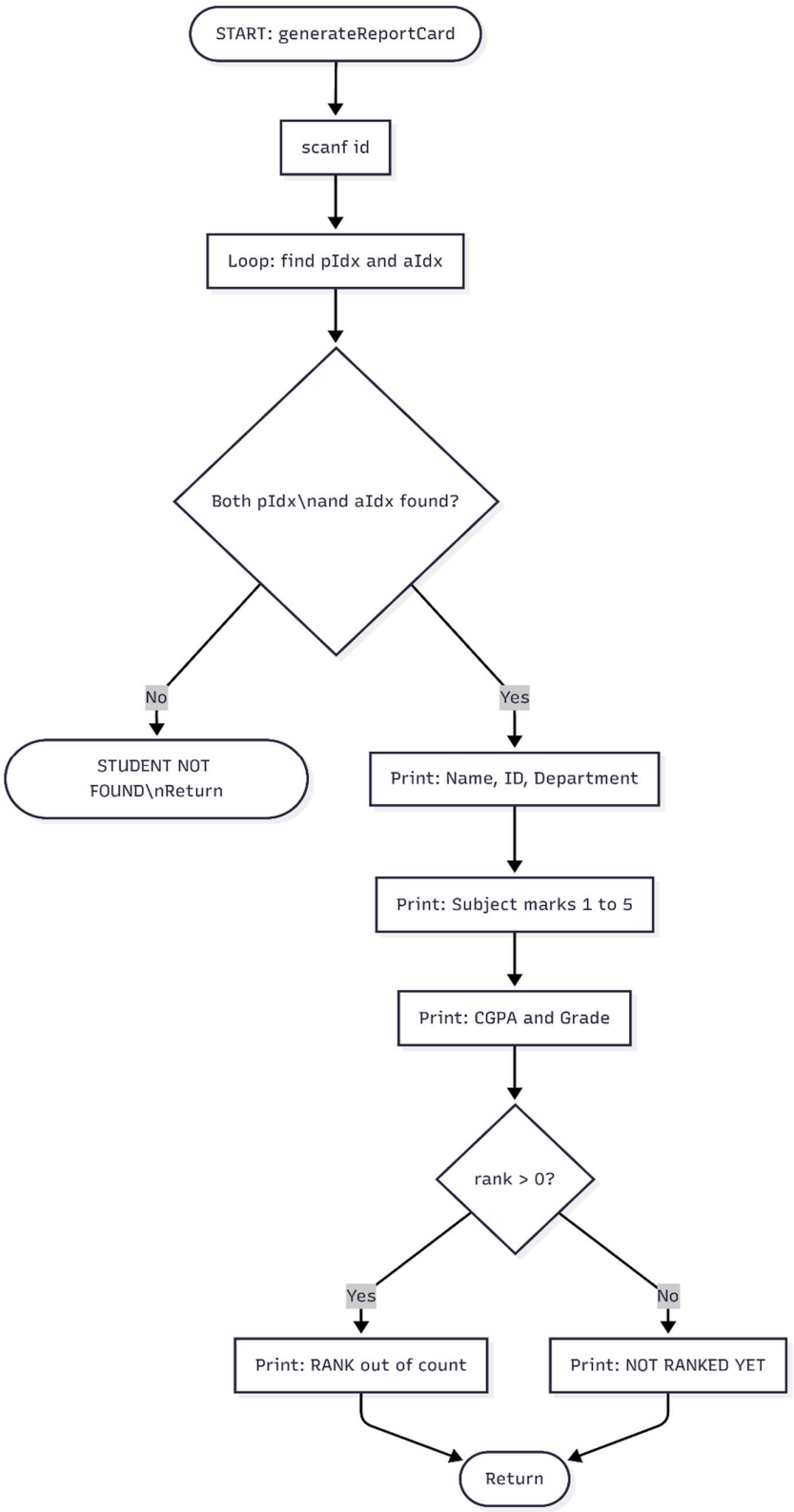


Fig: 2.13 (generateReportCard Function)

displayAnalytics Function

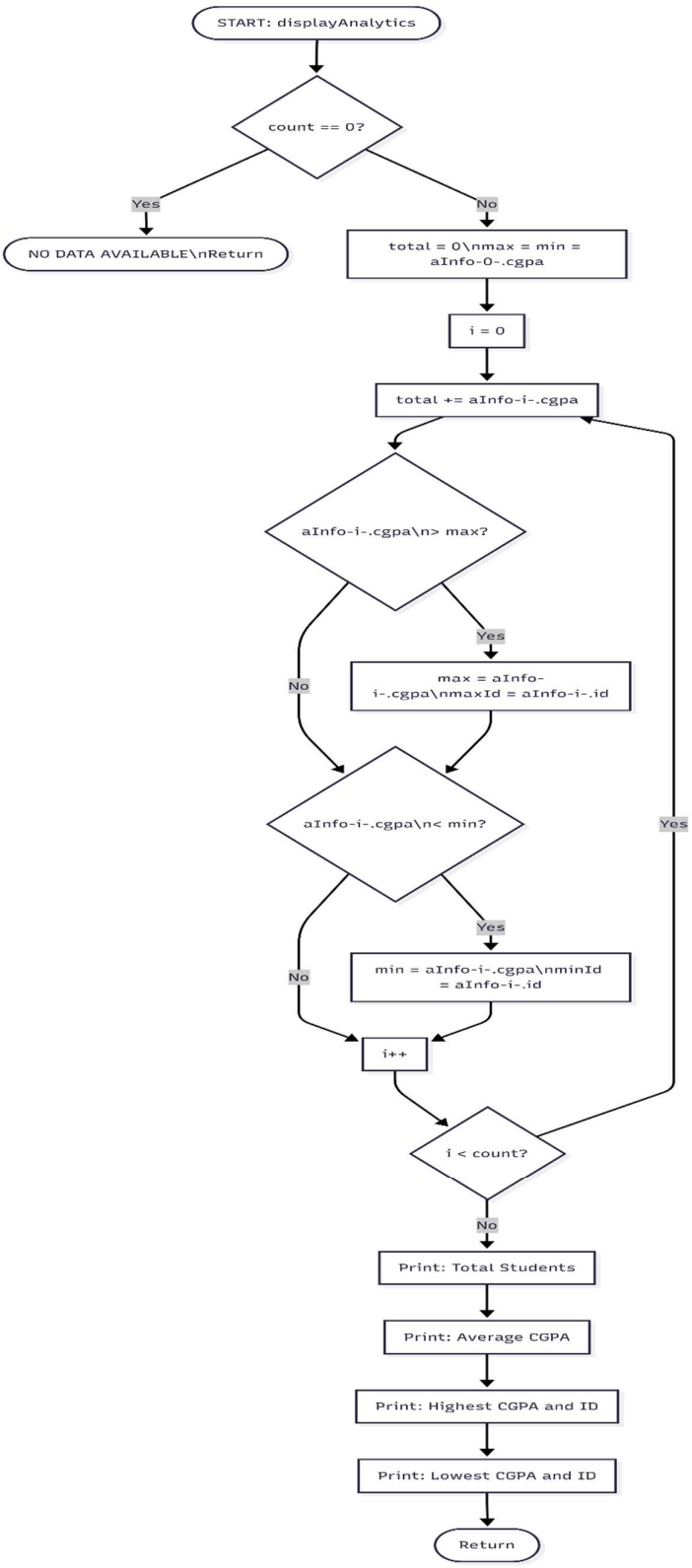


Fig: 2.14 (displayAnalytics Function)

updateStudentInDB Function

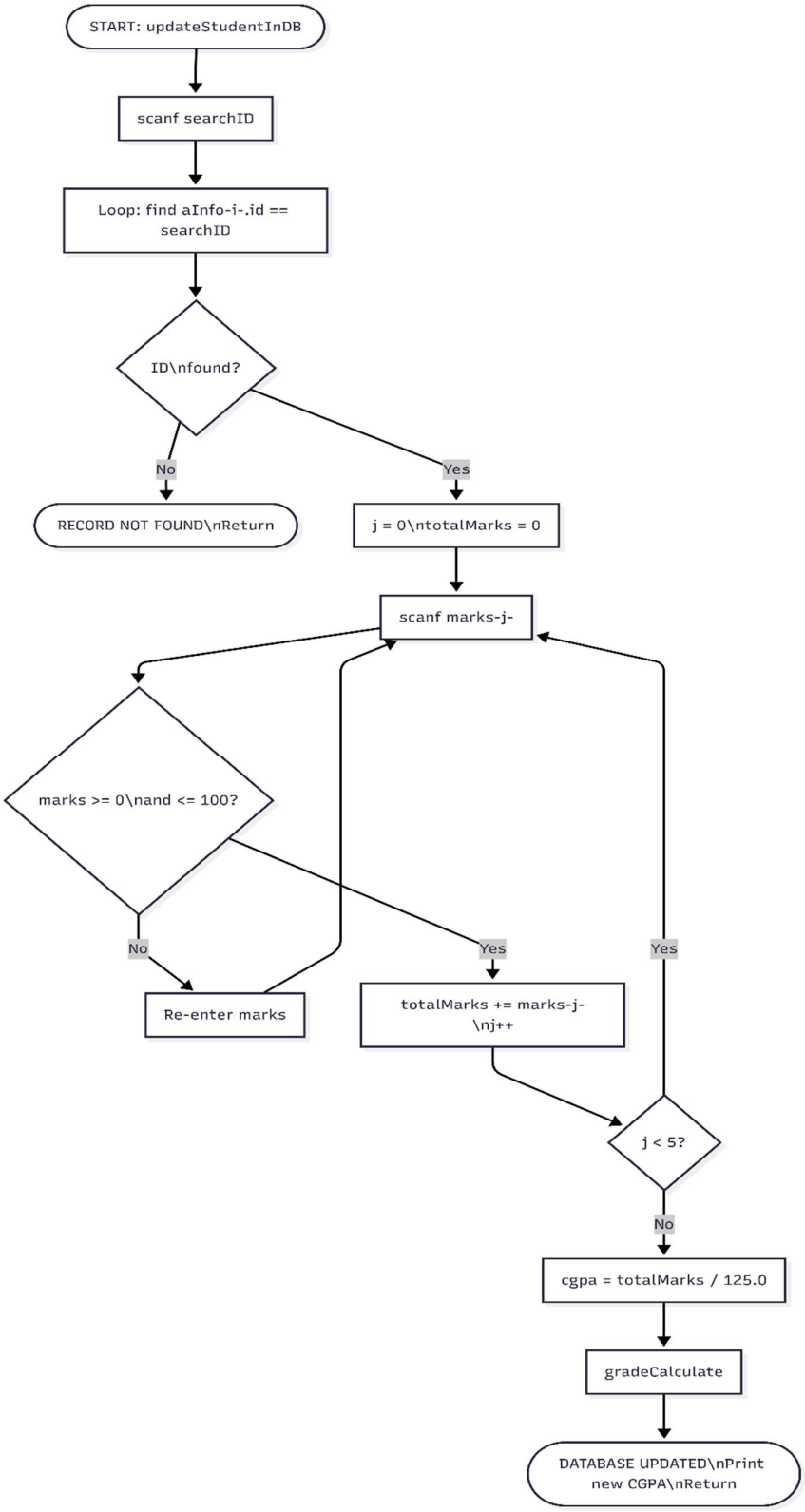


Fig: 2.15 (updateStudentInDB Function)

deleteStudentFromDB Function

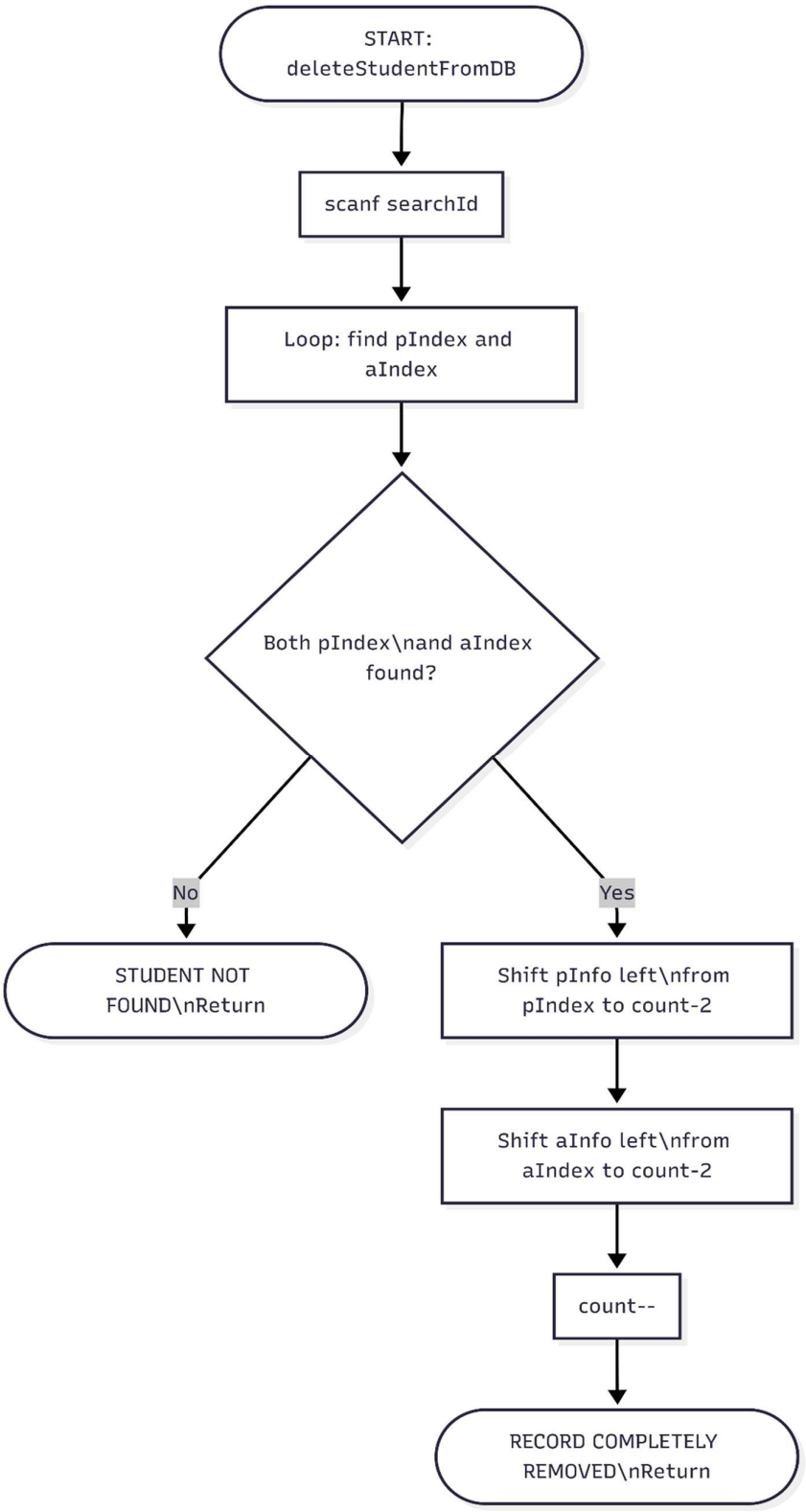


Fig: 2.16 (deleteStudentFromDB Function)

swapPaired & partition & sort Functions

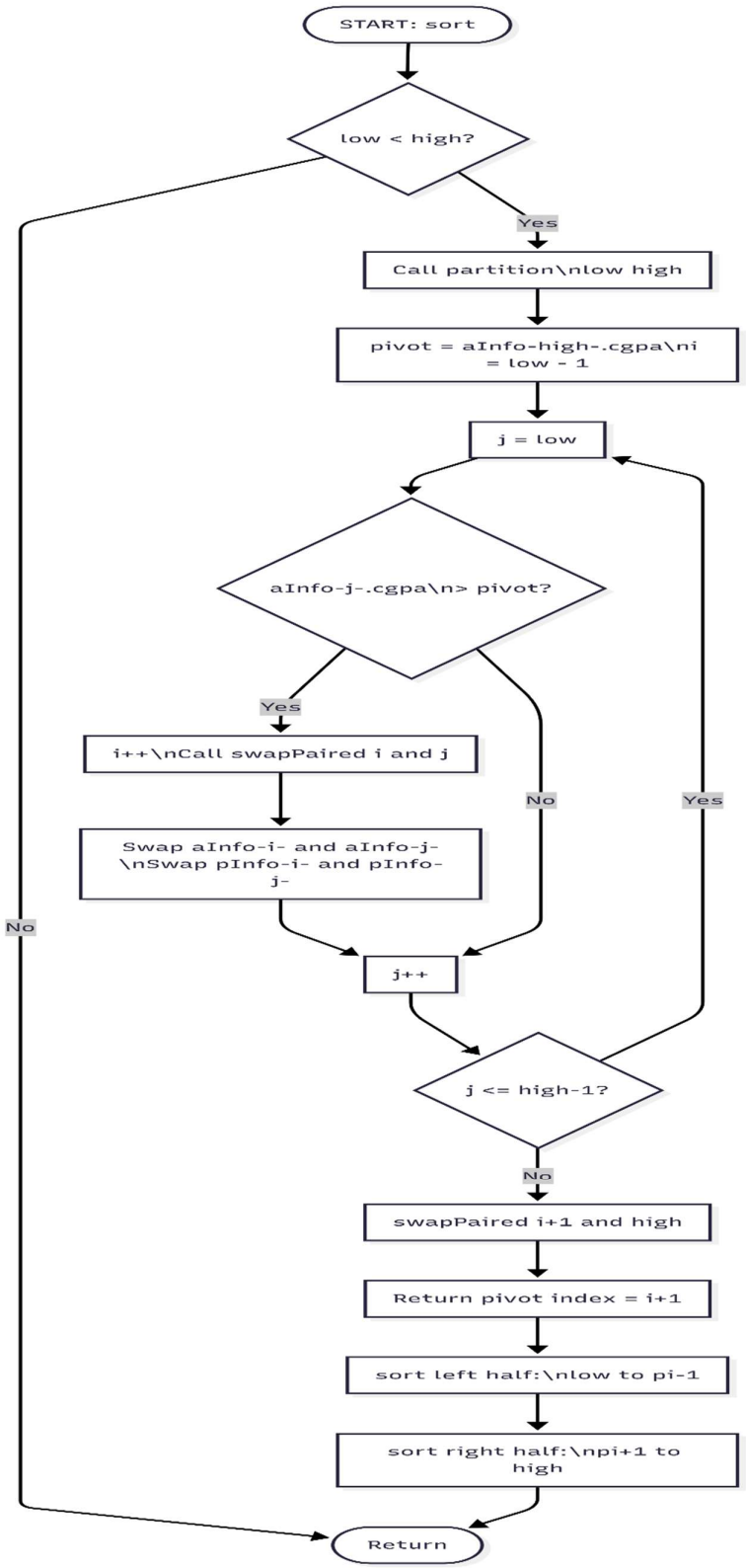


Fig: 2.17 (swapPaired & partition & sort Function)

rank Function

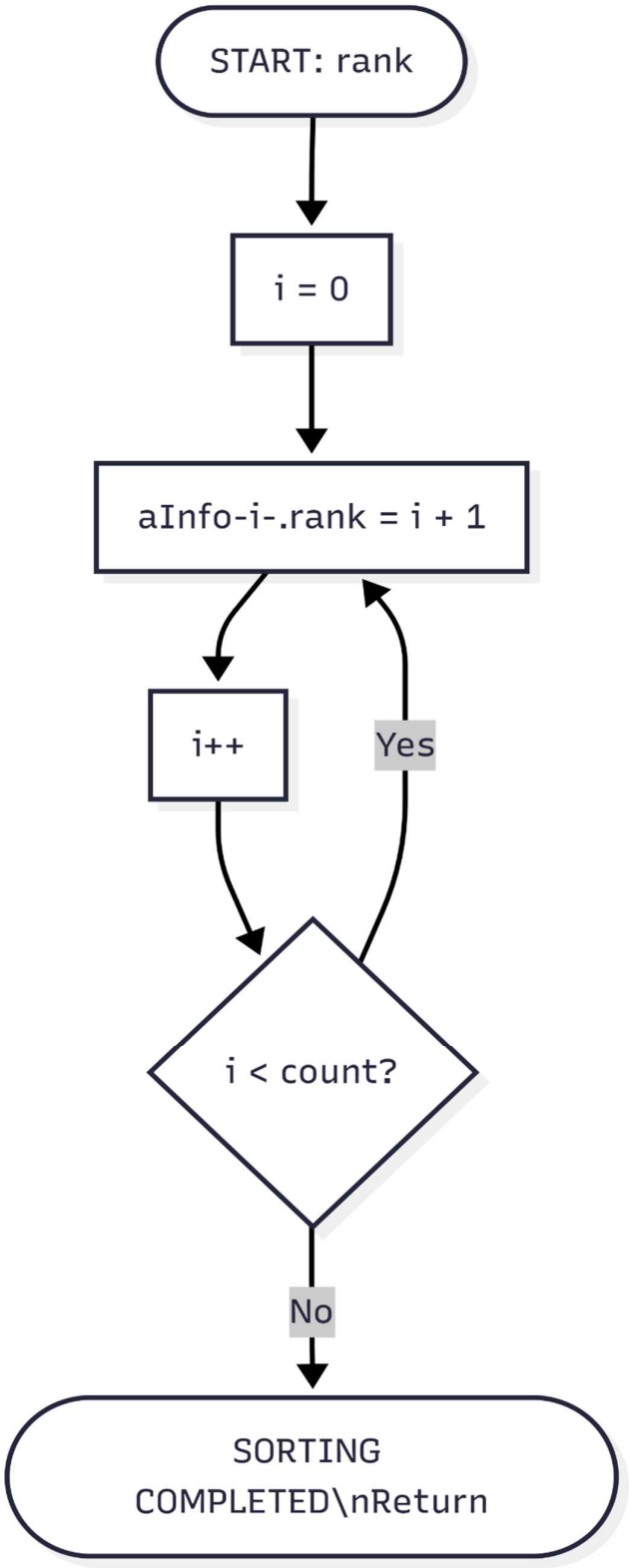


Fig: 2.18 (rank Function)

saveDatabase Function

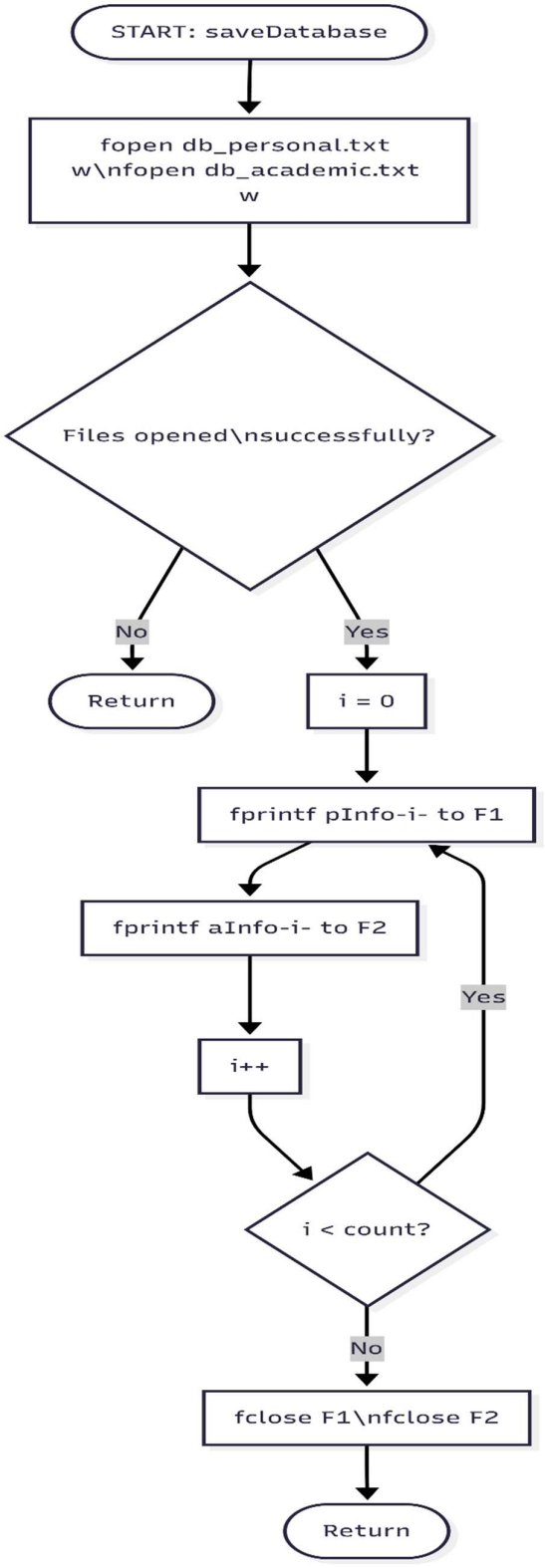


Fig: 2.19 (saveDatabase Function)

loadDatabase Function

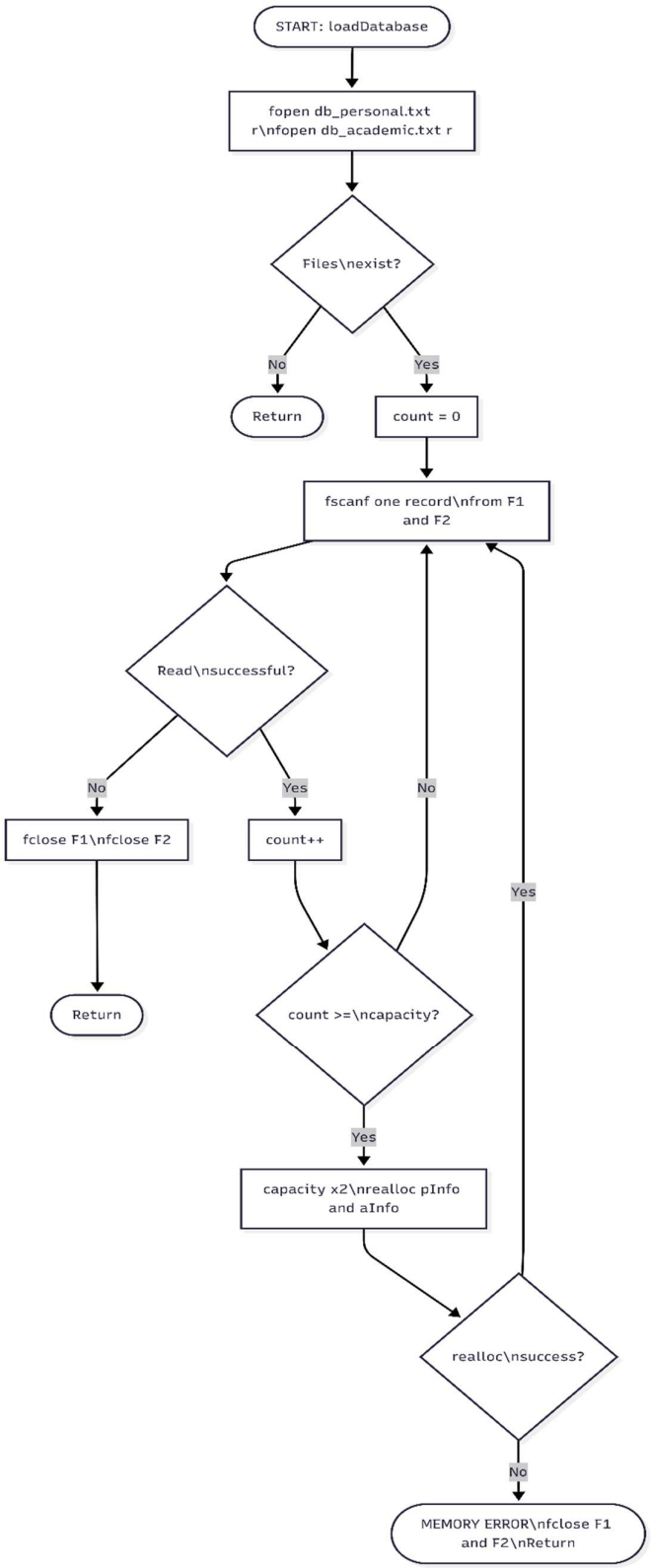


Fig: 2.20 (loadDatabase Function)

clearDatabase Function

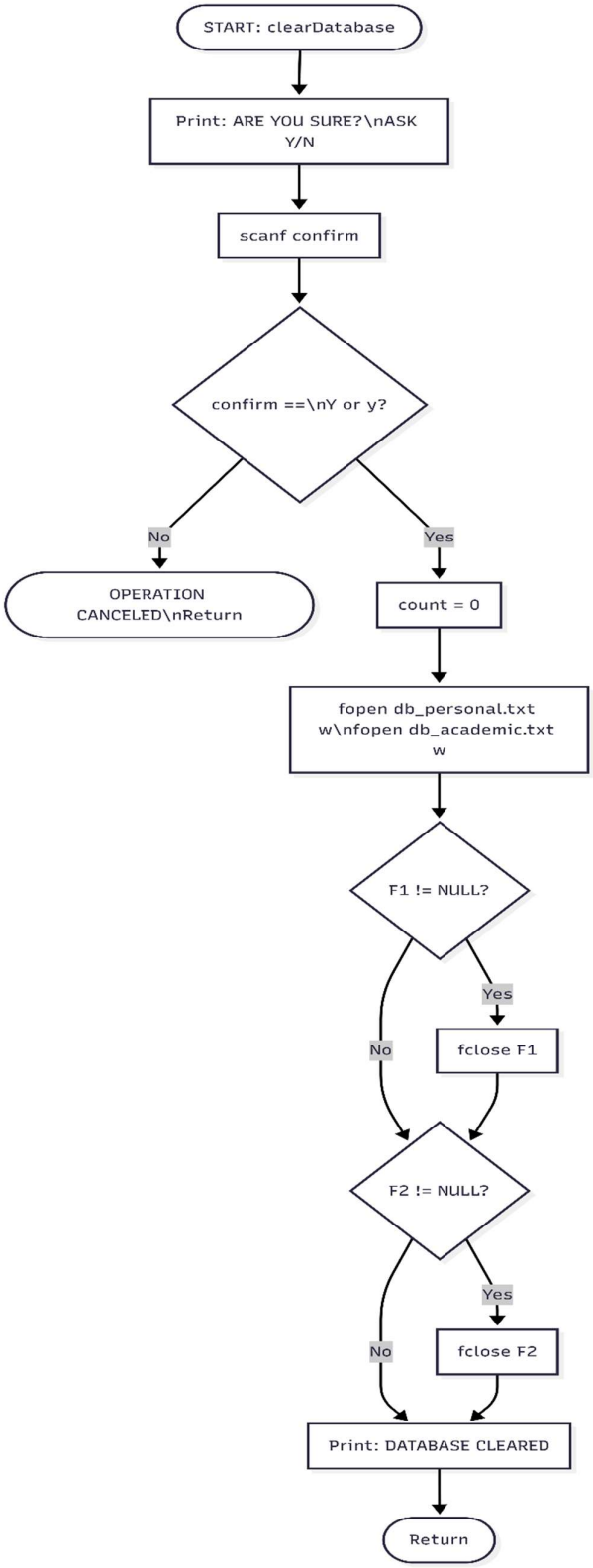


Fig: 2.21 (clearDatabase Function)

2.2 Overall Project Plan

The overall project plan adopts a systematic approach to develop the Academic Record Management System. Initially, core relational data structures and dynamic memory management were designed. Next, robust CRUD operations and algorithmic features like QuickSort for ranking were implemented. Finally, secure encrypted login, administrative account recovery, and persistent file I/O operations were integrated to ensure a secure, fully synchronized, and reliable terminal-based mini DBMS.

Chapter 3

Implementation and Results

This chapter details the practical implementation of the C-based Mini DBMS, highlighting the specific programming constructs, dynamic memory usage, and algorithms applied. Furthermore, it presents a comprehensive analysis of the system's performance and the final outcomes of its execution, including successful data synchronization and CRUD operations.

3.1 Implementation

Program Structures

Academic_Record_Management_System.c > loadDatabase(Database *)

```c
//  Personal Information Table
struct PersonalInfo {
    int id;
    char name[50];
    char dept[20];
};

//  Academic Information Table
struct AcademicInfo {
    int id;
    float marks[5];
    float cgpa;
    char grade[5];
    int rank;
};

//  Database Engine
struct Database {
    struct PersonalInfo *pInfo;
    struct AcademicInfo *aInfo;
    int count;
    int capacity;
};
```

Fig 3.1.1 Program Structure

Function Declarations

```
31
32  //  Functions declaration for this system
33  void passwordEncrypt(char *str);
34  void hiddenPass(char *pass);
35  int checkLogin();
36  void LockedSystem();
37  void changePass();
38  void gradeCalculate(float cgpa, char *grade);
39  void addStudentToDB(struct Database *db);
40  void importStudentFromFile(struct Database *db);
41  void updateStudentInDB(struct Database *db);
42  void deleteStudentFromDB(struct Database *db);
43  void displayJoinedData(struct Database *db);
44  void displayByDepartment(struct Database *db);
45  void displayAnalytics(struct Database *db);
46  void search(struct Database *db);
47  void generateReportCard(struct Database *db);
48  void swapPaired(struct Database *db, int i, int j);
49  int partition(struct Database *db, int low, int high);
50  void sort(struct Database *db, int low, int high);
51  void rank(struct Database *db);
52  void saveDatabase(struct Database *db);
53  void loadDatabase(struct Database *db);
54  void clearDatabase(struct Database *db);
55
```

Fig 3.1.2 Function Declarations

main Function

```
56
57  //  Main Function
58  int main() {
59
60  //  Security Check
61  if(checkLogin() == 0) {
62      LockedSystem();
63  }
64
65  //  Initialize Database
66  struct Database db;
67  db.capacity=5;
68  db.count=0;
69  db.pInfo=(struct PersonalInfo *)malloc(db.capacity * sizeof(struct PersonalInfo));
70  db.aInfo=(struct AcademicInfo *)malloc(db.capacity * sizeof(struct AcademicInfo));
71
72  if(db.pInfo==NULL || db.aInfo==NULL) {
73      printf("MEMORY ALLOCATION FAILED! EXITING.\n");
74      exit(1);
75  }
76
77  //  Load Information form Database
78  loadDatabase(&db);
79
80  int choice;
81  while(1) {
82      printf("\n\n        WELCOME TO OUR \"ACADEMIC RECORD MANAGEMENT SYSTEM WITH MINI DBMS\" PROJECT\n\n");
83      printf("01. ADD NEW STUDENT\n");
84      printf("02. IMPORT STUDENT INFORMATION FROM FILE\n");
85      printf("03. VIEW ALL STUDENT\n");
86      printf("04. VIEW DEPARTMENT WISE STUDENT\n");
87      printf("05. UPDATE STUDENT RECORD\n");
88      printf("06. DELETE STUDENT RECORD\n");
89      printf("07. SORT & RANK STUDENTS\n");
90      printf("08. VIEW CGPA ANALYTICS\n");
91      printf("09. ADVANCED SEARCH BY ID/NAME/CGPA\n");
92      printf("10. GENERATE REPORT CARD\n");
93      printf("11. SECURITY SETTINGS\n");
94      printf("12. CLEAR ALL INFORMATION FROM DATABASE\n");
95      printf("13. SAVE TO DATABASE & EXIT\n\n\n");
96      printf("CHOSE AN OPTION: ");
97
98      scanf("%d", &choice);
```

Fig 3.1.3 main Function

```
    switch(choice) {
        case 1:
            addStudentToDB(&db);
            break;
        case 2:
            importStudentFromFile(&db);
            break;
        case 3:
            displayJoinedData(&db);
            break;
        case 4:
            displayByDepartment(&db);
            break;
        case 5:
            updateStudentInDB(&db);
            break;
        case 6:
            deleteStudentFromDB(&db);
            break;
        case 7:
            if(db.count>0) {
                sort(&db, 0, db.count-1);
                rank(&db);
            }else {
                printf("NO DATA TO SORT..\n\n");
            }
            break;
        case 8:
            displayAnalytics(&db);
            break;
        case 9:
            search(&db);
            break;
        case 10:
            generateReportCard(&db);
            break;
        case 11:
            changePass();
            break;
        case 12:
            clearDatabase(&db);
            break;
        case 13:
            saveDatabase(&db);
            free(db.pInfo);
            free(db.aInfo);
            printf("DATABASE SAVED SUCCESSFULLY :)\n\n");
            return 0;
        default:
            printf("INVALID CHOICE :(\n\n");
    }
}
    return 0;
}
```

void clearDatabase(struct Database *db)

Fig 3.1.4 main Function

passwordEncrypt & hiddenPass Functions

```
155  //  SECURITY FEATURES
156  void passwordEncrypt(char *str) {
157      char key='K';
158
159      for(int i=0; str[i]!='\0'; i++) {
160          str[i]=str[i]^key;
161      }
162  }
163
164  void hiddenPass(char *pass) {
165      int i = 0;
166      char ch;
167      int maxLen = 49;
168
169      while(1) {
170          ch = getch();
171
172          if(ch == 13) {
173              pass[i] = '\0';
174              printf("\n");
175              break;
176          } else if(ch == 8) {
177              if(i > 0) {
178                  i--;
179                  printf("\b \b");
180              }
181          } else {
182              if(i < maxLen) {
183                  pass[i++] = ch;
184                  printf("*");
185              }
186          }
187      }
188  }
```

Fig 3.1.5 passwordEncrypt & hiddenPass Functions

checkLogin Function

```c
int checkLogin() {

    char savedPass[50], inputPass[50];

    FILE *file=fopen("password.bin", "rb");

    if(file==NULL) {
        file=fopen("password.bin", "wb");
        strcpy(savedPass, "admin123");
        passwordEncrypt(savedPass);
        fwrite(savedPass, sizeof(char), strlen(savedPass)+1, file);
        fclose(file);
        strcpy(savedPass, "admin123");
    }else {
        fread(savedPass, sizeof(char), 50, file);
        fclose(file);
        passwordEncrypt(savedPass);
    }

    int attempts=3;
    while(attempts>0) {
        printf("SECURITY LOGIN\n\n");
        printf("ENTER ADMIN PASSWORD: ");
        hiddenPass(inputPass);
        attempts--;

        if(strcmp(savedPass, inputPass)==0) {
            printf("LOGIN SUCCESSFUL!\n\n");
            return 1;
        }else {
            if(attempts>0) {
                printf("WRONG PASSWORD! TRY AGAIN.\n\n");
            }
        }
    }

    return 0;
}
```

Fig 3.1.6 checkLogin Function

LockedSystem Function

```
void LockedSystem() {
    printf("\nSYSTEM LOCKED!\n\n");

    char choice;
    printf("FORGOT PASSWORD? ANSWER THIS SECURITY QUESTION (Y/N): ");
    scanf(" %c", &choice);

    if(choice == 'Y' || choice == 'y') {
        char recAnswer[50];

        printf("\nSECURITY QUESTION: WHAT IS YOUR DEPARTMENT NAME?\n");
        printf("ENTER ANSWER: ");
        scanf("%49s", recAnswer);

        if(strcmp(recAnswer, "CSE") == 0) {
            printf("\nANSWER MATCHED!\n\n");
            printf("ENTER NEW ADMIN PASSWORD: ");

            char newPass[50];
            hiddenPass(newPass);

            passwordEncrypt(newPass);
            FILE *file = fopen("password.bin", "wb");
            if(file != NULL) {
                fwrite(newPass, sizeof(char), strlen(newPass)+1, file);
                fclose(file);
            }

            printf("\nPASSWORD RESET SUCCESSFULLY! PLEASE RESTART THE PROGRAM TO LOGIN.\n\n");
            exit(0);

        } else {
            printf("\nWRONG ANSWER! ACCESS DENIED.\n\n");
            exit(0);
        }

    } else {
        printf("\nPROGRAM TERMINATED!\n\n");
        exit(0);
    }
}
```

Fig 3.1.7 LockedSystem Function

changePass Function

```c
void changePass() {
    char oldPass[50], newPass[50], savedPass[50], securityAnswer[20];
    char choice;

    FILE *file = fopen("password.bin", "rb");
    if(file == NULL) {
        printf("PASSWORD FILE NOT FOUND! CANNOT CHANGE PASSWORD.\n\n");
        return;
    }
    fread(savedPass, sizeof(char), 50, file);
    fclose(file);
    passwordEncrypt(savedPass);

    printf("ENTER OLD PASSWORD: ");
    hiddenPass(oldPass);

    if(strcmp(savedPass, oldPass) == 0) {
        printf("PASSWORD MATCHED!\n\n");
    } else {
        printf("WRONG PASSWORD!\n");
        printf("FORGOT PASSWORD? ANSWER THIS SECURITY QUESTION (Y/N): ");

        scanf(" %c", &choice);

        if(choice == 'Y' || choice == 'y') {
            printf("WHAT IS YOUR UNIVERSITY NAME IN SHORT FORM: ");
            scanf("%19s", securityAnswer);

            if(strcmp(securityAnswer, "DIU") != 0 && strcmp(securityAnswer, "BRACU") != 0) {
                printf("WRONG ANSWER!!! ACCESS DENIED!!\n\n");
                return;
            }
            printf("CORRECT ANSWER!\n\n");
        } else {
            printf("ACCESS DENIED!!\n\n");
            return;
        }
    }

    printf("ENTER NEW PASSWORD: ");
    hiddenPass(newPass);
    passwordEncrypt(newPass);

    file = fopen("password.bin", "wb");
    if(file != NULL) {
        fwrite(newPass, sizeof(char), strlen(newPass)+1, file);
        fclose(file);
        printf("PASSWORD CHANGED SECURELY!\n\n");
    } else {
        printf("ERROR: COULD NOT SAVE NEW PASSWORD!\n\n");
    }
}
```

Fig 3.1.8 changePass Function

gradeCalculate Function

```
324    //  GRADING SYSTEM
325    void gradeCalculate(float cgpa, char *grade) {
326        if(cgpa==4.00) strcpy(grade, "A+");
327        else if(cgpa>=3.75) strcpy(grade, "A");
328        else if(cgpa>=3.50) strcpy(grade, "A-");
329        else if(cgpa>=3.25) strcpy(grade, "B+");
330        else if(cgpa>=3.00) strcpy(grade, "B");
331        else if(cgpa>=2.75) strcpy(grade, "C+");
332        else if(cgpa>=2.50) strcpy(grade, "C");
333        else if(cgpa>=2.25) strcpy(grade, "D+");
334        else if(cgpa>=2.00) strcpy(grade, "D");
335        else strcpy(grade, "F");
336    }
```

Fig 3.1.9 gradeCalculate Function

addStudentToDB Function

```c
//  MINI DBMS CRUD OPERATIONS
void addStudentToDB(struct Database *db) {

    if(db->count >= db->capacity) {
        db->capacity*=2;
        struct PersonalInfo *tmpP = realloc(db->pInfo, db->capacity * sizeof(struct PersonalInfo));
        struct AcademicInfo *tmpA = realloc(db->aInfo, db->capacity * sizeof(struct AcademicInfo));
        if(tmpP==NULL || tmpA==NULL) {
            printf("MEMORY ERROR! CANNOT ADD MORE STUDENTS.\n\n");
            db->capacity/=2;
            return;
        }
        db->pInfo=tmpP;
        db->aInfo=tmpA;
    }

    int tempID;
    printf("ENTER STUDENT ID: ");
    scanf("%d", &tempID);

    //Primary Key Validation
    for(int i=0; i<db->count; i++) {
        if(db->pInfo[i].id==tempID) {
            printf("ERROR:: ID ALREADY EXISTS IN DATABASE!\n\n");
            return;
        }
    }

    //Insert into Personal Information Table
    db->pInfo[db->count].id=tempID;
    printf("ENTER NAME: ");
    scanf("%49s", db->pInfo[db->count].name);
    printf("ENTER DEPARTMENT NAME: ");
    scanf("%19s", db->pInfo[db->count].dept);

    //Insert into Academic Information Table
    db->aInfo[db->count].id=tempID;
    float totalMarks=0;
    printf("ACADEMIC ENTRY\n\n");

    for(int i=0; i<5; i++) {

        do {
            printf("SUBJECT %d MARKS (0-100): ", i+1);
            scanf("%f", &db->aInfo[db->count].marks[i]);
        }while(db->aInfo[db->count].marks[i]<0 || db->aInfo[db->count].marks[i]>100);

        totalMarks+=db->aInfo[db->count].marks[i];
    }

    db->aInfo[db->count].cgpa=totalMarks/125.0;
    gradeCalculate(db->aInfo[db->count].cgpa, db->aInfo[db->count].grade);
    db->aInfo[db->count].rank=0;
    db->count++;
    printf("DATA SUCCESSFULLY INSERTED INTO BOTH TABLES!\n\n");
}
```

Fig 3.1.10 addStudentToDB Function

importStudentFromFile Function

```c
void importStudentFromFile(struct Database *db) {
    char fileName[50];
    printf("ENTER THE FILE NAME TO IMPORT THE STUDENT INFORMATIONS (like: students.txt): ");
    scanf("%49s", fileName);

    FILE *file=fopen(fileName, "r");
    if(file==NULL) {
        printf("FILE NOT FOUND! MAKE SURE THE FILE IS IN THE SAME FOLDER.\n\n");
        return;
    }

    int tempId, addedCount=0;
    char tempName[50], tempDept[50];
    float marks[5];

    while(fscanf(file, "%d %s %s %f %f %f %f %f", &tempId, tempName, tempDept, &marks[0], &marks[1], &marks[2], &marks[3], &marks[4])==8) {
        if(db->count >= db->capacity) {
            db->capacity*=2;
            struct PersonalInfo *tmpP = realloc(db->pInfo, db->capacity * sizeof(struct PersonalInfo));
            struct AcademicInfo *tmpA = realloc(db->aInfo, db->capacity * sizeof(struct AcademicInfo));
            if(tmpP==NULL || tmpA==NULL) {
                printf("MEMORY ERROR! CANNOT ADD MORE STUDENTS.\n\n");
                db->capacity/=2;
                fclose(file);
                return;
            }
            db->pInfo=tmpP;
            db->aInfo=tmpA;
        }

        int skip=0;
        for(int i=0; i<db->count; i++) {
            if(db->pInfo[i].id==tempId) {
                printf("SKIPPING ID %d: IT'S ALREADY EXISTED IN DATABASE.\n\n", tempId);
                skip=1;
                break;
            }
        }

        if(skip) continue;

        //  Data inserting into Personal Information Table
        db->pInfo[db->count].id=tempId;
        strcpy(db->pInfo[db->count].name, tempName);
        strcpy(db->pInfo[db->count].dept, tempDept);

        //  Data inserting into Academic Information Table
        db->aInfo[db->count].id=tempId;
        float totalMarks=0;
        for(int i=0; i<5; i++) {
            db->aInfo[db->count].marks[i]=marks[i];
            totalMarks+=marks[i];
        }

        db->aInfo[db->count].cgpa=totalMarks/125.0;
        gradeCalculate(db->aInfo[db->count].cgpa, db->aInfo[db->count].grade);
        db->aInfo[db->count].rank=0;

        db->count++;
        addedCount++;
    }

    fclose(file);
    printf("THE INFORMATION OF THE STUDENTS HAS SUCCESSFULLY IMPORTED TO DATABASE FROM THE FILE :0\n\n");
    printf("%d NEW STUDENTS ARE ADDED TO THE DATABASE :0\n\n", addedCount);
}
```

Fig 3.1.11 importStudentFromFile Function

updateStudentInDB Function

```
461
462  void updateStudentInDB(struct Database *db) {
463      int searchID, found=0;
464      printf("ENTER ID TO UPDATE ACADEMIC RECORDS: ");
465      scanf("%d", &searchID);
466
467      for(int i=0; i<db->count; i++) {
468          if(db->aInfo[i].id==searchID) {
469              found=1;
470              float totalMarks=0;
471              printf("UPDATING MARKS FOR %d ID.....\n\n", searchID);
472
473              for(int j=0; j<5; j++) {
474                  printf("NEW SUBJECT %d MARKS: ", j+1);
475                  scanf("%f", &db->aInfo[i].marks[j]);
476
477                  totalMarks+=db->aInfo[i].marks[j];
478              }
479
480              db->aInfo[i].cgpa=totalMarks/125.0;
481              gradeCalculate(db->aInfo[i].cgpa, db->aInfo[i].grade);
482              printf("DATABASE UPDATED SUCCESSFULLY :) NEW CGPA: %.2f\n", db->aInfo[i].cgpa);
483              break;
484          }
485      }
486
487      if(!found) printf("RECORD NOT FOUND IN DATABASE :(\n\n");
488  }
489
```

Fig 3.1.12 updateStudentInDB Function

deleteStudentFomDB Function

```
490  void deleteStudentFromDB(struct Database *db) {
491      int searchId, pIndex=-1, aIndex=-1;
492      printf("ENTER ID TO DELETE: ");
493      scanf("%d", &searchId);
494
495      //Finding Student ID in both Personal and Academic Table
496      for(int i=0; i<db->count; i++) {
497          if(db->pInfo[i].id==searchId) pIndex=i;
498          if(db->aInfo[i].id==searchId) aIndex=i;
499      }
500
501      if(pIndex!=-1 && aIndex!=-1) {
502          for(int i=pIndex; i<db->count-1; i++) {
503              db->pInfo[i]=db->pInfo[i+1];
504          }
505          for(int i=aIndex; i<db->count-1; i++) {
506              db->aInfo[i]=db->aInfo[i+1];
507          }
508
509          db->count--;
510          printf("RECORD COMPLETELY REMOVED FROM DATABASE.\n\n");
511      }else printf("STUDENT NOT FOUND :(\n\n");
512  }
513
```

Fig 3.1.13 deleteStudentFomDB Function

displayJoinedData Function

```
514  //  ANALYTICS & SEARCH
515  void displayJoinedData(struct Database *db) {
516
517      if(db->count==0) {
518          printf("DATABASE IS EMPTY!!\n");
519          return;
520      }
521
522      printf("\nID\tNAME\t\tDEPT\tCGPA\tGRADE\tRANK\n");
523
524      for(int i=0; i<db->count; i++) {
525          int currentID=db->pInfo[i].id;
526
527          for(int j=0; j<db->count; j++) {
528              if(db->aInfo[j].id==currentID) {
529                  printf("%d\t%-10s\t%s\t%.2f\t%s\t%d\n", db->pInfo[i].id, db->pInfo[i].name,
530                      db->pInfo[i].dept, db->aInfo[j].cgpa, db->aInfo[j].grade, db->aInfo[j].rank);
531                  break;
532              }
533          }
534      }
535  }
536
```

Fig 3.1.14 displayJoinedData Function

displayByDepartment Function

```c
void displayByDepartment(struct Database *db) {
    if(db->count == 0) {
        printf("DATABASE IS EMPTY :(\n");
        return;
    }

    char searchDept[20];
    printf("\nENTER DEPARTMENT NAME (like: CSE, EEE, BBA): ");
    scanf("%19s", searchDept);

    struct DeptData {
        int id;
        char name[50];
        float cgpa;
        char grade[5];
    };

    struct DeptData *deptList = (struct DeptData *)malloc(db->count * sizeof(struct DeptData));
    int deptCount = 0;

    for(int i = 0; i < db->count; i++) {
        if(strcasecmp(db->pInfo[i].dept, searchDept) == 0) {
            deptList[deptCount].id = db->pInfo[i].id;
            strcpy(deptList[deptCount].name, db->pInfo[i].name);

            for(int j = 0; j < db->count; j++) {
                if(db->aInfo[j].id == db->pInfo[i].id) {
                    deptList[deptCount].cgpa = db->aInfo[j].cgpa;
                    strcpy(deptList[deptCount].grade, db->aInfo[j].grade);
                    break;
                }
            }
            deptCount++;
        }
    }

    if(deptCount == 0) {
        printf("NO STUDENTS FOUND IN %s DEPARTMENT.\n\n", searchDept);
        free(deptList);
        return;
    }

    for(int i = 0; i < deptCount - 1; i++) {
        for(int j = 0; j < deptCount - i - 1; j++) {
            if(deptList[j].cgpa < deptList[j+1].cgpa) {
                struct DeptData temp = deptList[j];
                deptList[j] = deptList[j+1];
                deptList[j+1] = temp;
            }
        }
    }

    printf("\n\t\tSTUDENTS IN %s DEPARTMENT\n\n", searchDept);
    printf("DEPT RANK\tID\tNAME\t\tCGPA\tGRADE\n\n");

    for(int i = 0; i < deptCount; i++) {
        printf("%d\t\t%d\t%-10s\t%.2f\t%s\n", i + 1, deptList[i].id,
               deptList[i].name, deptList[i].cgpa, deptList[i].grade);
    }

    printf("\n");
    free(deptList);
}
```

Fig 3.1.15 displayByDepartment Function

displayAnalytics Function

```
598
599  void displayAnalytics(struct Database *db) {
600
601      if(db->count==0) {
602          printf("NO DATA AVAILABLE :(\n\n");
603          return;
604      }
605
606      float total=0, max=db->aInfo[0].cgpa, min=db->aInfo[0].cgpa;
607      int maxId=db->aInfo[0].id, minId=db->aInfo[0].id;
608
609      for(int i=0; i<db->count; i++) {
610          total+=db->aInfo[i].cgpa;
611
612          if(db->aInfo[i].cgpa>max) {
613              max=db->aInfo[i].cgpa;
614              maxId=db->aInfo[i].id;
615          }
616          if(db->aInfo[i].cgpa<min) {
617              min=db->aInfo[i].cgpa;
618              minId=db->aInfo[i].id;
619          }
620      }
621
622      printf("\n\n\t\tGPA STATISTICS\n\n");
623      printf("TOTAL STUDENTS: %d\n", db->count);
624      printf("AVERAGE CGPA  : %.2f\n", total/db->count);
625      printf("HIGHEST CGPA  : %.2f (ID: %d)\n", max, maxId);
626      printf("LOWEST CGPA   : %.2f (ID: %d)\n\n", min, minId);
627  }
628
```

Fig 3.1.16 displayAnalytics Function

search Function

```c
void search(struct Database *db) {
    if(db->count==0) return;

    int option;
    printf("\n\t\tSELECT OPTION\n\n");
    printf("1. SEARCH BY ID\n");
    printf("2. SEARCH BY NAME\n");
    printf("3. SEARCH BY CGPA\n\n");
    printf("ENTER YOUR CHOICE: ");
    scanf("%d", &option);

    if(option==1) {
        int id, found=0;
        printf("ENTER ID: ");
        scanf("%d", &id);

        for(int i=0; i<db->count; i++) {
            if(db->pInfo[i].id==id) {
                printf("\nFOUND THE STUDENT!\n\n");
                printf("NAME        : %s\n", db->pInfo[i].name);
                printf("ID          : %d\n", db->pInfo[i].id);
                printf("DEPARTMENT  : %s\n", db->pInfo[i].dept);
                printf("CGPA        : %.2f\n", db->aInfo[i].cgpa);
                printf("GRADE       : %s\n\n", db->aInfo[i].grade);
                found=1;
                break;
            }
        }
        if(!found) printf("NOT FOUND :(\n\n");
    }else if(option==2) {
        char searchName[50];
        int found=0;
        printf("ENTER NAME: ");
        scanf("%49s", searchName);

        for(int i=0; i<db->count; i++) {
            if(strcmp(db->pInfo[i].name, searchName)==0) {
                printf("\nFOUND THE STUDENT!\n\n");
                printf("NAME        : %s\n", db->pInfo[i].name);
                printf("ID          : %d\n", db->pInfo[i].id);
                printf("DEPARTMENT  : %s\n", db->pInfo[i].dept);
                printf("CGPA        : %.2f\n", db->aInfo[i].cgpa);
                printf("GRADE       : %s\n\n", db->aInfo[i].grade);
                found=1;
                break;
            }
        }
        if(!found) printf("NO STUDENT FOUND WITH THIS NAME :(\n\n");
    }else if(option==3) {
        float min, max;
        int found=0;
        printf("ENTER RANGE (e.g. 3.0 4.0): ");
        scanf("%f %f", &min, &max);

        for(int i=0; i<db->count; i++) {
            if(db->aInfo[i].cgpa>=min && db->aInfo[i].cgpa<=max) {
                printf("\nFOUND THE STUDENT!\n\n");
                printf("NAME        : %s\n", db->pInfo[i].name);
                printf("ID          : %d\n", db->pInfo[i].id);
                printf("DEPARTMENT  : %s\n", db->pInfo[i].dept);
                printf("CGPA        : %.2f\n", db->aInfo[i].cgpa);
                printf("GRADE       : %s\n\n", db->aInfo[i].grade);
                found=1;
            }
        }
        if(!found) printf("NO STUDENTS IN THIS RANGE :(\n\n");
    }
}
```

Fig 3.1.17 search Function

generateReportCard Function

```c
//  Report Card Generator
void generateReportCard(struct Database *db) {
    int id;
    printf("ENTER STUDENT ID FOR PEPORT CARD: ");
    scanf("%d", &id);

    int pIdx=-1, aIdx=-1;
    for(int i=0; i<db->count; i++) {
        if(db->pInfo[i].id==id) pIdx=i;
        if(db->aInfo[i].id==id) aIdx=i;
    }

    if(pIdx!=-1 && aIdx!=-1) {
        printf("        \n\nOFFICIAL ACADEMIC TRANSCRIPT\n\n");
        printf("NAME         : %-20s\n", db->pInfo[pIdx].name);
        printf("ID           : %d\n", db->pInfo[pIdx].id);
        printf("DEPARTMENT   : %-20s\n", db->pInfo[pIdx].dept);
        printf("MARKS        : SUBJECT-1(%.0f)\tSUBJECT-2(%.0f)\tSUBJECT-3(%.0f)\tSUBJECT-4(%.0f)\tSUBJECT-5(%.0f)\n",
            db->aInfo[aIdx].marks[0], db->aInfo[aIdx].marks[1], db->aInfo[aIdx].marks[2],
            db->aInfo[aIdx].marks[3], db->aInfo[aIdx].marks[4]);

        printf("CGPA         : %.2f\n", db->aInfo[aIdx].cgpa);
        printf("GRADE        : %s\n", db->aInfo[aIdx].grade);
        if(db->aInfo[aIdx].rank>0) {
            printf("RANK         : %d OUT OF %d\n\n", db->aInfo[aIdx].rank, db->count);
        }else printf("RANK       : NOT RANKED YET :(\n\n");
    }else printf("STUDENT NOT FOUND IN DATABASE :(\n\n");
}
```

Fig 3.1.18 generateReportCard Function

swapPaired & partition & sort & rank Functions

```c
//  SORT & RANK
void swapPaired(struct Database *db, int i, int j) {
    struct AcademicInfo tempA = db->aInfo[i];
    db->aInfo[i] = db->aInfo[j];
    db->aInfo[j] = tempA;

    struct PersonalInfo tempP = db->pInfo[i];
    db->pInfo[i] = db->pInfo[j];
    db->pInfo[j] = tempP;
}

int partition(struct Database *db, int low, int high) {
    float pivot = db->aInfo[high].cgpa;
    int i = low - 1;

    for(int j = low; j <= high - 1; j++) {
        if(db->aInfo[j].cgpa > pivot) {
            i++;
            swapPaired(db, i, j);
        }
    }
    swapPaired(db, i + 1, high);
    return (i + 1);
}

void sort(struct Database *db, int low, int high) {
    if(low < high) {
        int pi = partition(db, low, high);
        sort(db, low, pi - 1);
        sort(db, pi + 1, high);
    }
}

void rank(struct Database *db) {
    for(int i = 0; i < db->count; i++) {
        db->aInfo[i].rank = i + 1;
    }
    printf("SORTING COMPLETED! DATABASE TABLES ARE RELATIONALLY MAPPED.\n\n");
}
```

Fig 3.1.19 swapPaired & partition & sort & rank Functions

saveDatabase Function

```
764  //  DBMS FILE I/O
765  void saveDatabase(struct Database *db) {
766      FILE *F1=fopen("db_personal.txt", "w");
767      FILE *F2=fopen("db_academic.txt", "w");
768
769      if(F1==NULL || F2==NULL) return;
770
771      for(int i=0; i<db->count; i++) {
772          // Save Personal Information Table
773          fprintf(F1, "%d %s %s\n", db->pInfo[i].id, db->pInfo[i].name, db->pInfo[i].dept);
774          // Save Academic Information Table
775          fprintf(F2, "%d %f %f %f %f %f %s %d\n", db->aInfo[i].id, db->aInfo[i].marks[0],
776              db->aInfo[i].marks[1], db->aInfo[i].marks[2], db->aInfo[i].marks[3], db->aInfo[i].marks[4],
777              db->aInfo[i].cgpa, db->aInfo[i].grade, db->aInfo[i].rank);
778      }
779      fclose(F1);
780      fclose(F2);
781  }
782
```

Fig 3.1.20 saveDatabase Function

loadDatabase Function

```
781  void loadDatabase(struct Database *db) {
782      FILE *F1=fopen("db_personal.txt", "r");
783      FILE *F2=fopen("db_academic.txt", "r");
784
785      if(F1==NULL || F2==NULL) return;
786
787      db->count=0;
788      while(fscanf(F1, "%d %s %s", &db->pInfo[db->count].id, db->pInfo[db->count].name, db->pInfo[db->count].dept)!=EOF) {
789          fscanf(F2, "%d %f %f %f %f %f %s %d", &db->aInfo[db->count].id, &db->aInfo[db->count].marks[0],
790              &db->aInfo[db->count].marks[1], &db->aInfo[db->count].marks[2], &db->aInfo[db->count].marks[3],
791              &db->aInfo[db->count].marks[4], &db->aInfo[db->count].cgpa, db->aInfo[db->count].grade, &db->aInfo[db->count].rank);
792          db->count++;
793
794          if(db->count >= db->capacity) {
795              db->capacity*=2;
796              struct PersonalInfo *tmpP = realloc(db->pInfo, db->capacity * sizeof(struct PersonalInfo));
797              struct AcademicInfo *tmpA = realloc(db->aInfo, db->capacity * sizeof(struct AcademicInfo));
798              if(tmpP==NULL || tmpA==NULL) {
799                  printf("MEMORY ERROR! CANNOT ADD MORE STUDENTS.\n\n");
800                  db->capacity/=2;
801                  fclose(F1);
802                  fclose(F2);
803                  return;
804              }
805              db->pInfo=tmpP;
806              db->aInfo=tmpA;
807          }
808      }
809      fclose(F1);
810      fclose(F2);
811  }
```

Fig 3.1.21 loadDatabase Function

clearDatabase Function

```
810
811  void clearDatabase(struct Database *db) {
812      char confirm;
813      printf("ARE YOU SURE WANT TO CLEAR ALL DATA? THIS CAN'T BE UNDONE! (Y/N): ");
814      scanf(" %c", &confirm);
815
816      if(confirm=='Y' || confirm=='y') {
817          db->count=0;
818          FILE *F1=fopen("db_personal.txt", "w");
819          FILE *F2=fopen("db_academic.txt", "w");
820
821          if(F1!=NULL) fclose(F1);
822          if(F2!=NULL) fclose(F2);
823
824          printf("DATABASE HAS BEEN CLEARED SUCCESSFULLY :0\n\n");
825      }else printf("OPERATION CANCELED. YOUR DATA IS SAFE :0\n\n");
826  }
```

Fig 3.1.21 clearDatabase Function

3.2 Results and Discussion

In this section, the core operational outputs of the Academic Record Management System are presented, along with technical discussions detailing the underlying C programming logic.

3.2.1 Security & Access Control Module

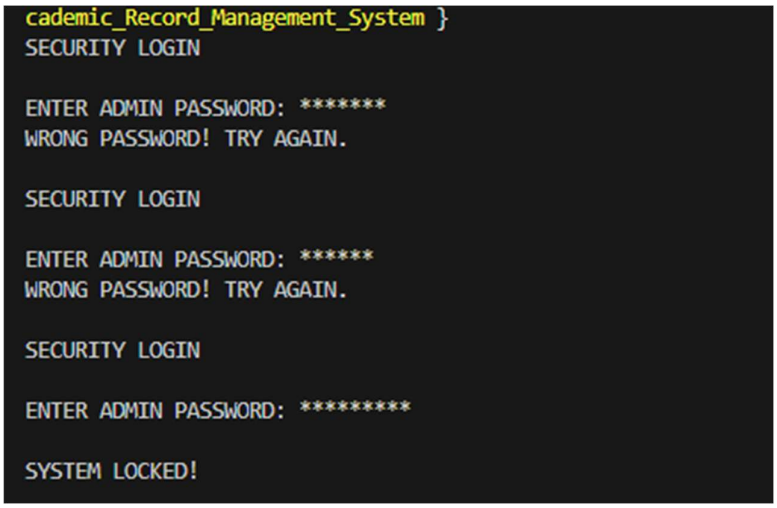


Fig 3.2.1.1 Authentication and Failure Handling

Discussion on Authentication and Failure Handling: The system employs a robust authentication mechanism via the checkLogin() function. It uses getch() for masked password entry to ensure privacy. If a user enters an incorrect password, the system triggers a failure logic that decrements the total allowed attempts (3 attempts by default). This is handled using a while loop and a counter variable. It is preventing unauthorized brute-force access.

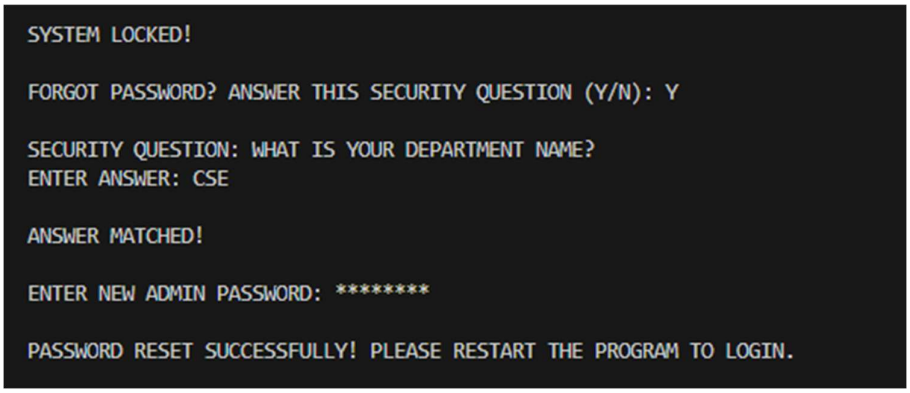


Fig 3.2.1.2 System Lockout and Recovery

Discussion on System Lockout and Recovery: Once the maximum failed attempts are reached, the LockedSystem() function is invoked, which completely freezes the administrative operations. To regain access, the system introduces a Recovery Option. This works by requiring a predefined 'Master Recovery Key' (embedded securely in the code logic). If the admin provides the correct recovery key, the system initiates the changePass() function, allowing the user to reset the password and update the password.bin file after XOR encryption. This ensures that even if a password is forgotten or the system is locked, there is a secure way to bypass the lockout without data loss.

3.2.2 Administrative Main Menu

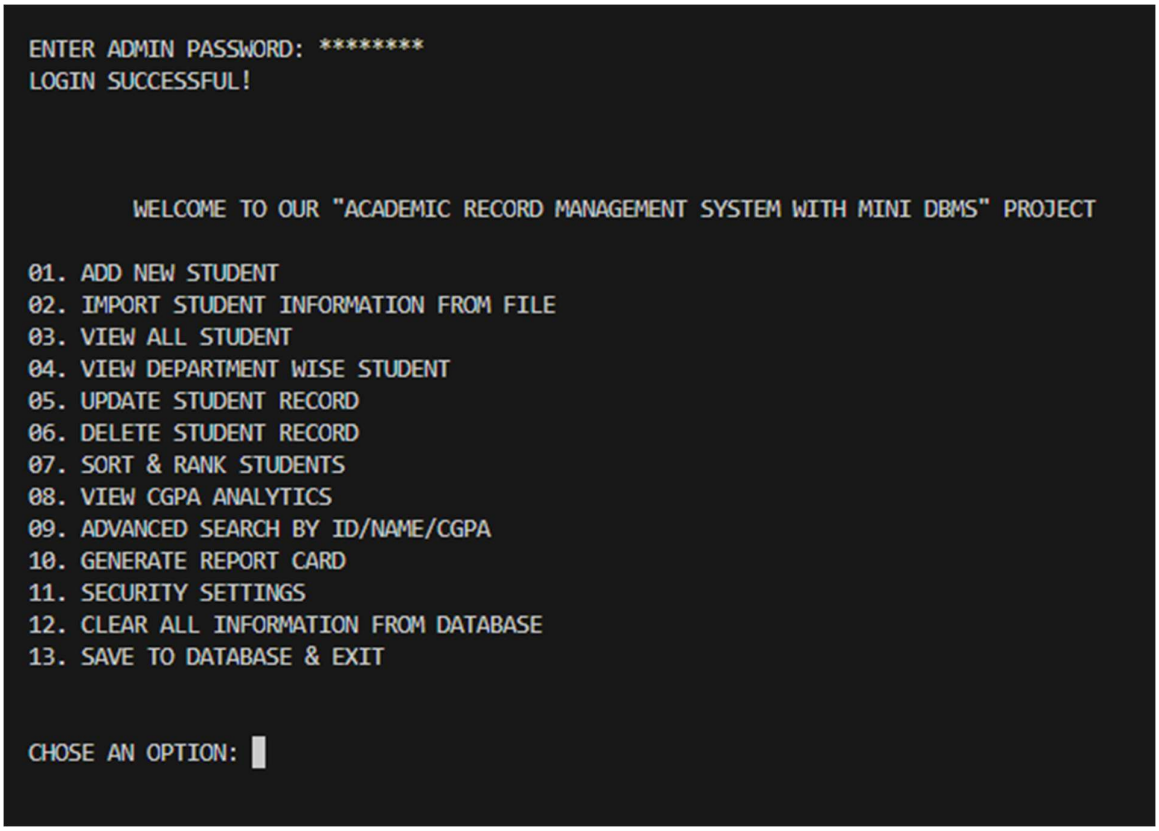


Fig 3.2.2 Administrative Main Menu

Discussion on Dashboard Overview: Upon successful authentication, the user is directed to the Main Menu. This is the 'Command Center' of the software, implemented using an infinite while(1) loop and a switch-case control structure. The menu provides a comprehensive overview of all available modules:

- Data Management: Options for Adding, Updating, and Deleting records.
- Information Retrieval: Options for Viewing all data, searching by ID, or filtering by Department.
- Analytics & Performance: Tools for Sorting (QuickSort), Academic Analytics, and Ranking.
- System Settings: Options for clearing the database or changing the access password.

3.2.3 Adding a New Student Record

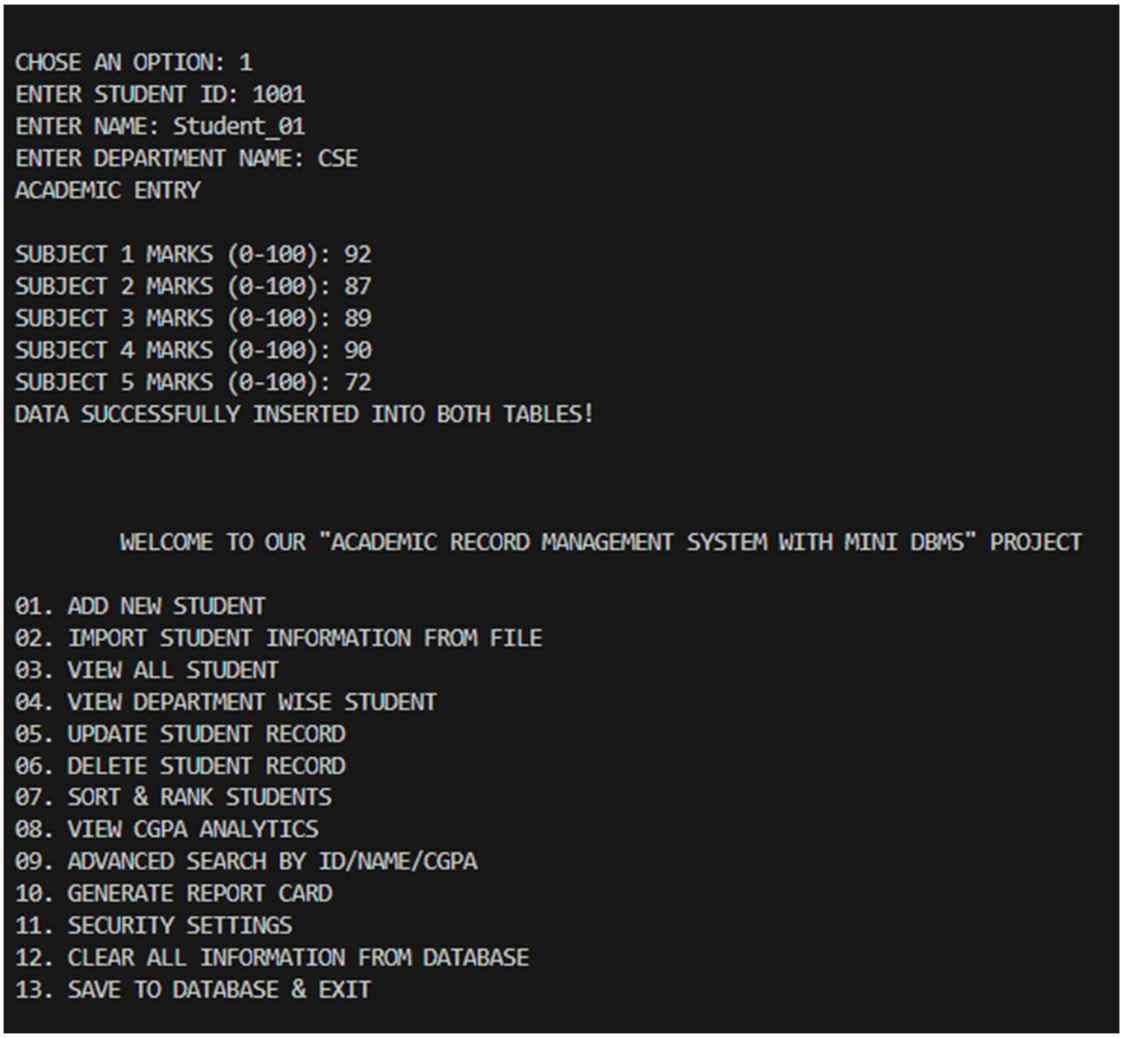


Fig 3.2.3 Adding a New Student Record

Discussion on Data Insertion Logic: This output illustrates the initial data entry process managed by the addStudentToDB() function. When the administrator inputs a new student's ID, Name, Department, and raw subject marks, the system seamlessly captures this information. Behind the scenes, the function first verifies if the current dynamic memory array has sufficient capacity. If the limit is reached, it intelligently expands the memory block using the realloc() function to prevent overflow. Once the memory is secured, the system stores the data into the respective PersonalInfo and AcademicInfo structures. Concurrently, it processes the 5 subject marks through the gradeCalculate() function to automatically compute the correct CGPA and assign the corresponding letter grade before finalizing the entry.

3.2.4 Data Import from External Files

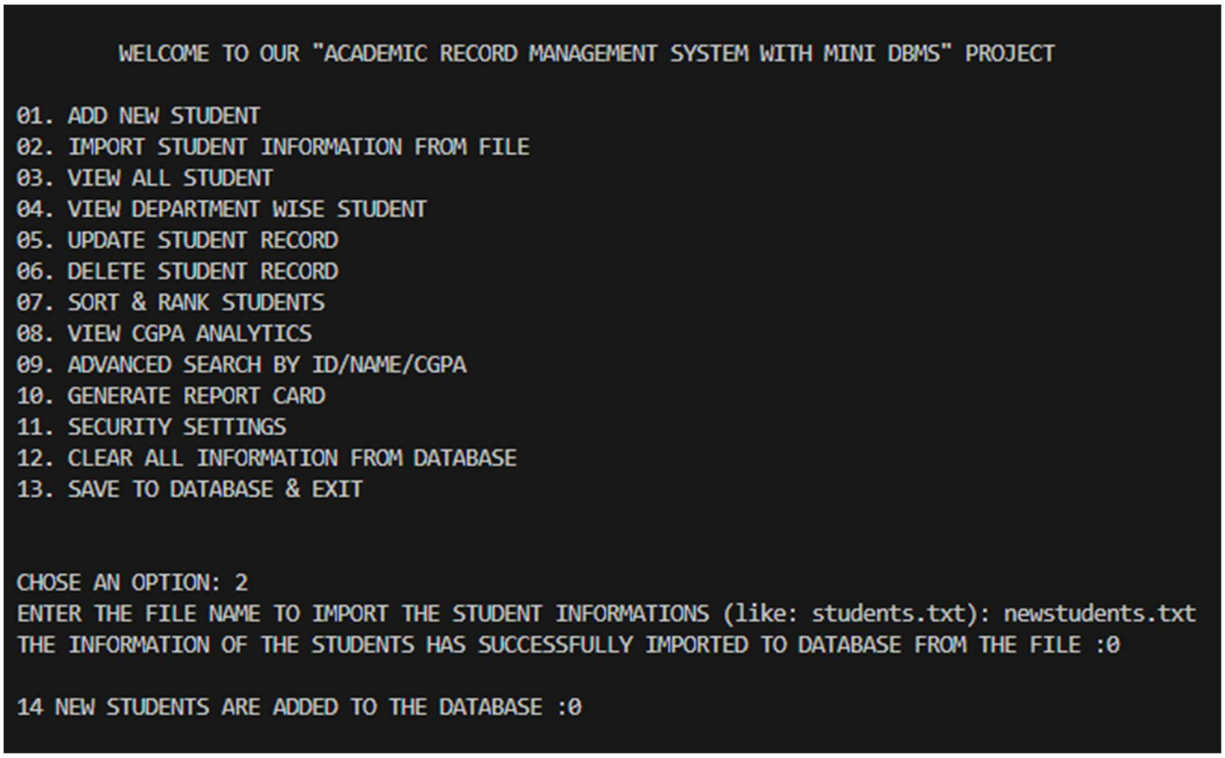


Fig 3.2.4 Data Import from External Files

Discussion on File-Based Data Import Logic: This output demonstrates the system's bulk data entry capability, managed by the importStudentFromFile() function. Instead of manual one-by-one entry, this module allows administrators to load multiple records simultaneously. The program opens and reads from external raw data files (import_personal.txt and import_academic.txt) utilizing standard C File I/O operations (fopen, fscanf).

During the import process, the system actively monitors the active memory limits. If the incoming data exceeds the current array size, it intelligently utilizes realloc() to dynamically expand the memory allocation without crashing. After successfully fetching the records, the system processes the raw marks to automatically calculate the CGPA and Letter Grades. Finally, these imported records are merged with the existing database arrays and saved back to the primary storage files, ensuring seamless and efficient bulk data integration.

3.2.5 Displaying All Student Records (Relational View)

```
       WELCOME TO OUR "ACADEMIC RECORD MANAGEMENT SYSTEM WITH MINI DBMS" PROJECT

01. ADD NEW STUDENT
02. IMPORT STUDENT INFORMATION FROM FILE
03. VIEW ALL STUDENT
04. VIEW DEPARTMENT WISE STUDENT
05. UPDATE STUDENT RECORD
06. DELETE STUDENT RECORD
07. SORT & RANK STUDENTS
08. VIEW CGPA ANALYTICS
09. ADVANCED SEARCH BY ID/NAME/CGPA
10. GENERATE REPORT CARD
11. SECURITY SETTINGS
12. CLEAR ALL INFORMATION FROM DATABASE
13. SAVE TO DATABASE & EXIT


CHOSE AN OPTION: 3

ID      NAME            DEPT    CGPA    GRADE   RANK
1012    Student_12      CSE     3.77    A       1
1008    Student_8       BBA     3.67    A-      2
1003    Student_3       BBA     3.60    A-      3
1014    Student_14      BBA     3.55    A-      4
1005    Student_5       CSE     3.47    B+      5
1001    Student_01      CSE     3.44    B+      6
1007    Student_7       EEE     3.40    B+      7
1010    Student_10      ENG     3.35    B+      8
1002    Student_2       EEE     3.30    B+      9
1013    Student_13      EEE     3.27    B+      10
1009    Student_9       CSE     3.05    B       11
1015    Student_15      ENG     2.95    C+      12
1006    Student_6       LLB     2.87    C+      13
1004    Student_4       ENG     2.80    C+      14
1011    Student_11      LLB     2.80    C+      15
```

Fig 3.2.5 Displaying All Student Records (Relational View)

Discussion on Relational Data Retrieval Logic: This output is generated by the displayJoinedData() function, which serves as the primary data retrieval and viewing module of the system. The function works by iterating through the stored records and simulating a relational database "JOIN" operation. It seamlessly fetches the personal details (Name, Department) from the PersonalInfo structure and merges them with the dynamically calculated academic results (Marks, CGPA, Grade) from the AcademicInfo structure, presenting them in a unified tabular format.

Notably, the "Rank" column in this output displays accurate positional rankings (e.g., 1, 2, 3) because the dataset was pre-processed using the system's sorting module (QuickSort algorithm) prior to invoking the view function. This demonstrates that the system effectively retains the updated state of the variables in memory. The output is cleanly presented using formatted string specifiers in C to ensure a professional and easy-to-read console interface.

3.2.6 View Department Wise Student

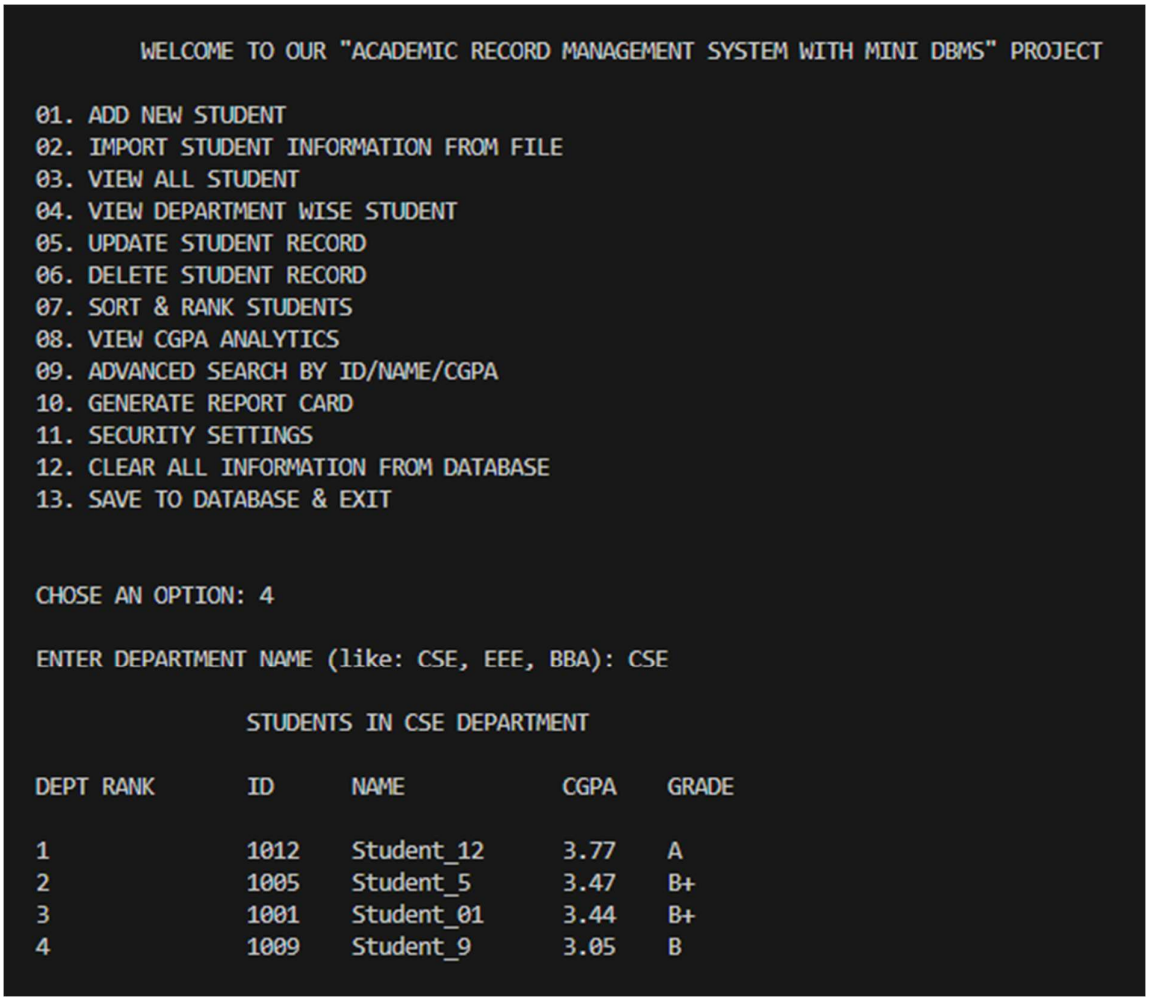


Fig 3.2.6 View Department Wise Student

Discussion on Department-Wise Filtering Logic: This output showcases the system's data filtering capability, executed through the displayByDepartment() function. When the administrator inputs a specific department name (e.g., "CSE"), the program iterates through the PersonalInfo array and uses the strcasecmp() or strcmp() function to identify matching strings. Once a department match is confirmed, the system uses the unique Student ID as a foreign key to perform a relational lookup within the AcademicInfo structure. This mechanism efficiently extracts and displays only the filtered subset of students belonging to that particular department, along with their respective academic records.

3.2.7 Update Student Record

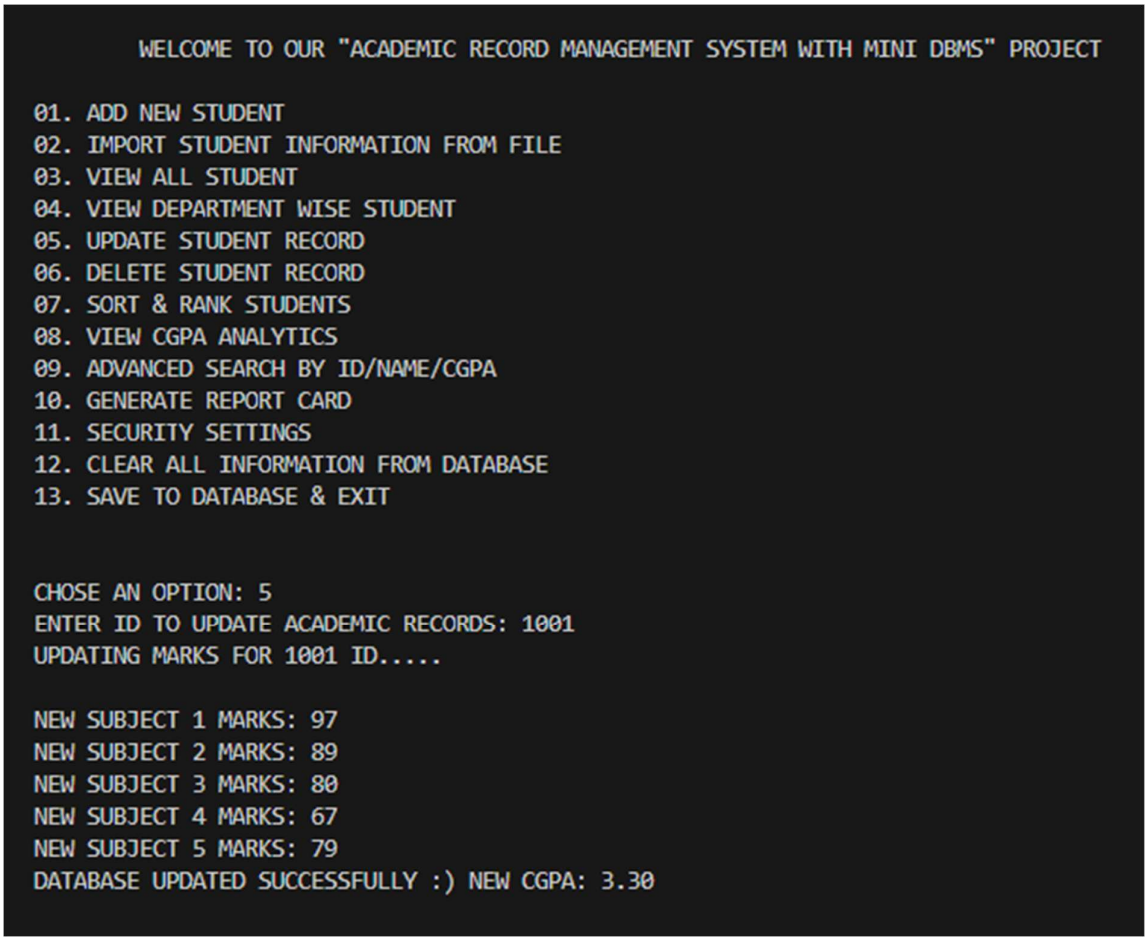


Fig 3.2.7 Update Student Record

Discussion on Data Modification Logic: This output illustrates the system's dynamic record modification capability, executed through the updateStudentInDB() function. The technical workflow begins with a linear search algorithm targeting a specific Student ID provided by the user.

For instance, as demonstrated in the specific output above, when the administrator intends to modify a record and enters the target ID 1001, the system successfully searches and locates the corresponding memory block in the dynamically allocated structure array. It then prompts the user to input the revised academic data. Upon receiving the new subject marks (97, 89, 80, 67, 79), the system directly overwrites the previous data in the aInfo structure. Crucially, the system automatically triggers the gradeCalculate() function with these newly inputted marks to instantly recalculate and display the updated result (New CGPA: 3.30). Finally, the system ensures data persistence by immediately overwriting the db_personal.txt and db_academic.txt files with the updated arrays, synchronizing the active memory with local storage.

3.2.8 Sorting and Ranking Students

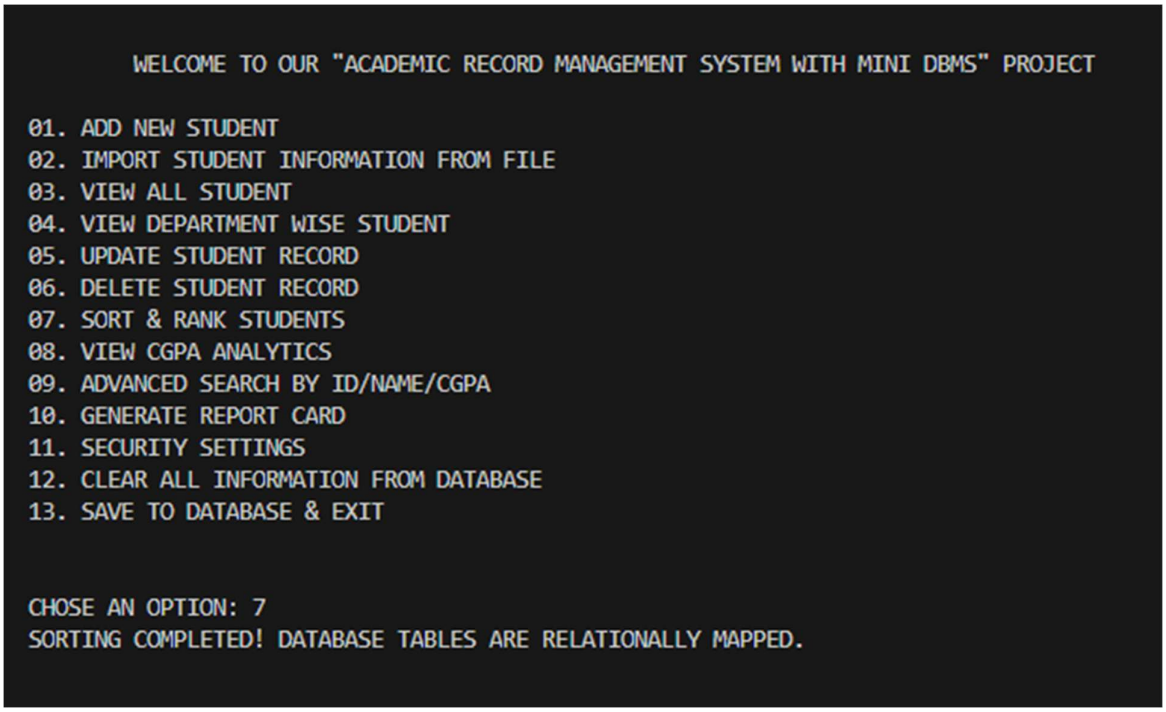


Fig 3.2.8 Sorting and Ranking Students

Discussion on Sorting Algorithm and Relational Integrity: This output demonstrates the successful execution of the system's core algorithmic feature: the sorting and ranking module. When Option 7 is triggered, the program initiates a highly efficient QuickSort algorithm (with an average time complexity of $O(n \log n)$) to internally reorder the dynamically allocated memory arrays. The primary sorting key utilized here is the CGPA, arranging the student records in descending order to mathematically determine their merit positions.

The most critical technical challenge addressed in this module is maintaining Relational Data Mapping. Because the personal data (PersonalInfo) and academic data (AcademicInfo) are housed in completely separate structural arrays, sorting only the academic records would corrupt the database by mismatching CGPAs with the wrong student names. To overcome this, the algorithm integrates a synchronized swapping mechanism. Whenever the sorting logic swaps two academic records based on their CGPA, it simultaneously swaps their corresponding personal records at the exact same indices.

The console message *"SORTING COMPLETED! DATABASE TABLES ARE RELATIONALLY MAPPED"* confirms that this synchronized operation was successful. Consequently, the data integrity remains intact, and the active memory is fully prepared to accurately display positional ranks (e.g., 1, 2, 3) across all other viewing and reporting modules.

3.2.9 Academic Analytics and Performance Overview

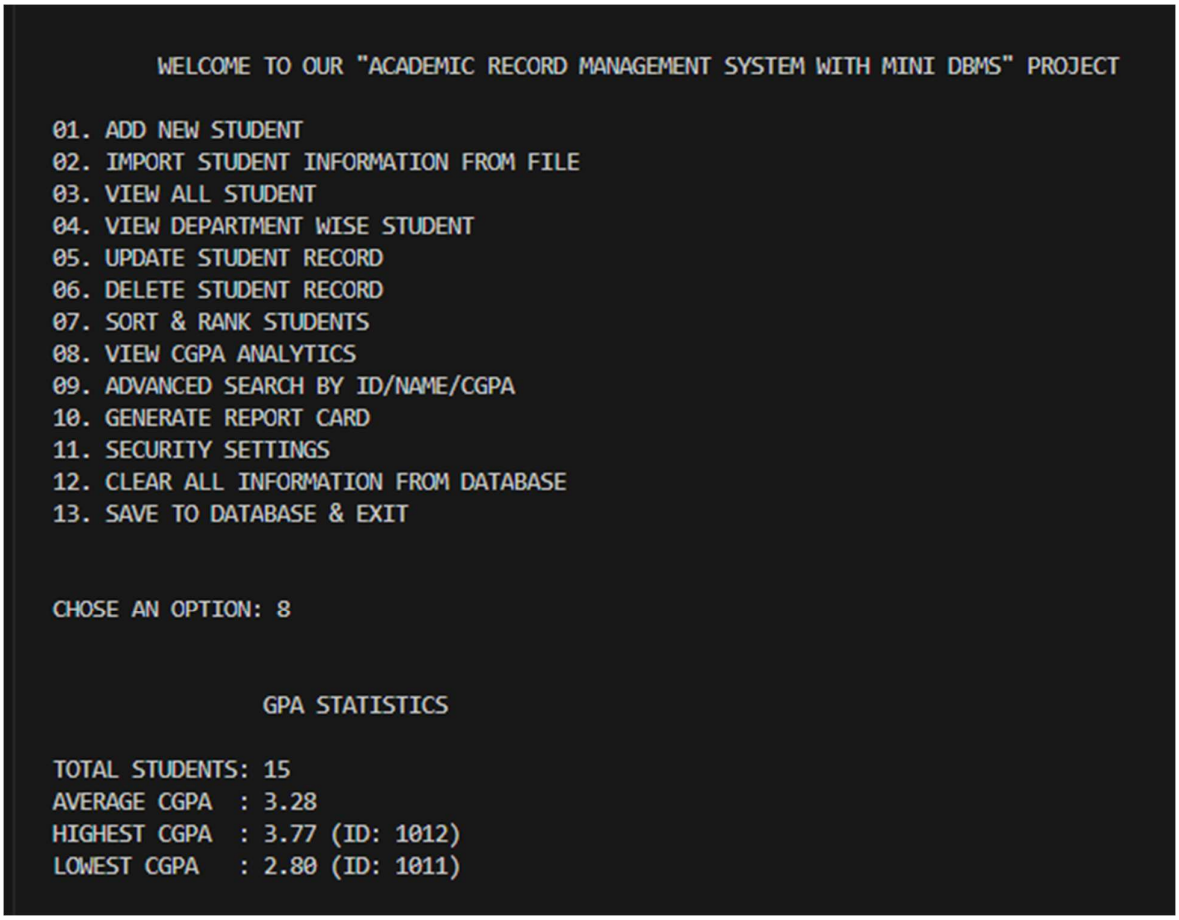


Fig 3.2.9 Academic Analytics and Performance Overview

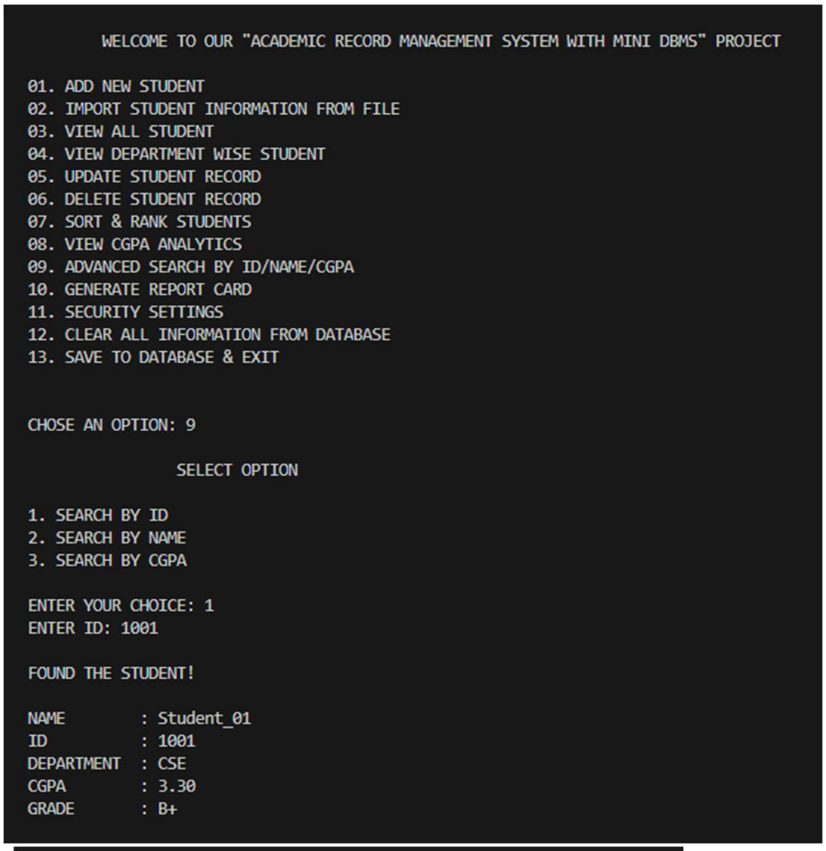
Discussion on Statistical Computation Logic: This output illustrates the system's capability to perform aggregate data analysis through the viewCGPAAnalytics() (or equivalent) function. Rather than displaying individual records, this module calculates and presents a macro-level statistical summary of the entire student database.

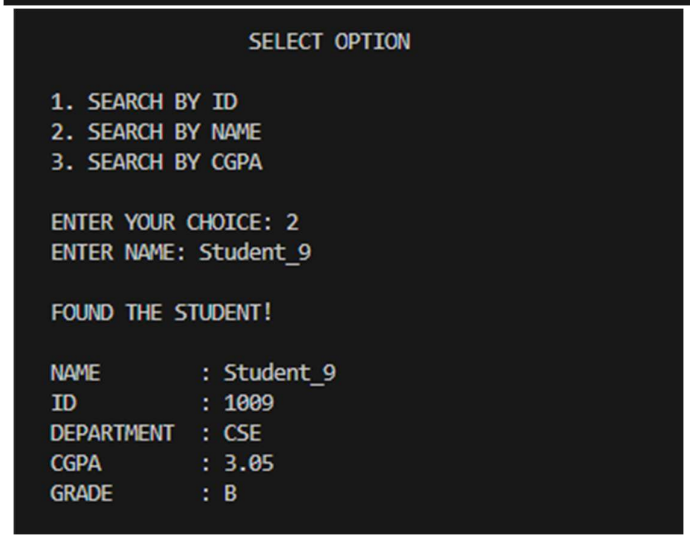
The underlying algorithm executes a single linear traversal ($O(n)$ time complexity) across the dynamically allocated AcademicInfo structure array. During this traversal, the system computes several key metrics simultaneously:

1. Total Students: Retrieves the current value of the global or local counter variable tracking the total active records (e.g., 15 students).
2. Average CGPA: Accumulates the CGPA of all students into a sum variable and divides it by the total number of students to calculate the mean (e.g., Average: 3.28).
3. Highest and Lowest CGPA: Utilizes a standard min-max finding algorithm. It compares each student's CGPA against the current max and min variables. When a new highest or lowest value is found, the system updates the respective variable and temporarily stores the associated Student ID as a reference.

As demonstrated in the output, the system successfully identifies the highest CGPA (3.77 belonging to ID: 1012) and the lowest CGPA (2.80 belonging to ID: 1011). This feature provides administrators with immediate, actionable insights into the overall academic performance of the institution without requiring manual calculation.

3.2.10 Advanced Multi-Parameter Search Module





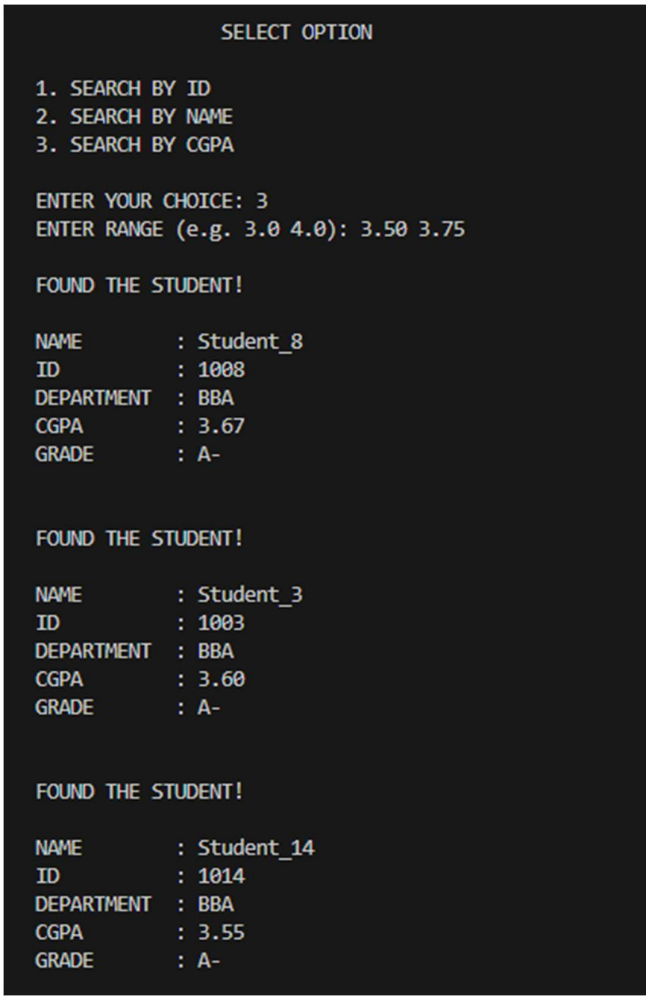


Fig 3.2.10 Advance Searching

Discussion on Multi-Criteria Query Logic: These outputs demonstrate the system's robust querying capabilities, handled by the advanced search module. This module mimics the behavior of a standard SQL WHERE clause, allowing administrators to retrieve specific subsets of data based on three distinct criteria:

1. Search by ID (Exact Integer Match): As seen in the first output, when the user inputs a specific ID (e.g., 1001), the system executes a linear search algorithm ($O(n)$ time complexity) across the PersonalInfo array. Since IDs are unique primary keys, the search terminates immediately upon finding the exact integer match, fetching and displaying the mapped relational data.
2. Search by Name (String Comparison): The second output illustrates string-based searching. The system captures the target name (e.g., Student_9) and iterates through the database using

C string handling functions like strcmp() or strcasecmp(). Upon a successful string match, it retrieves the corresponding personal and academic details.

3. Search by CGPA (Dynamic Range Query): The third output showcases the most advanced query type—a range-based filter. Instead of an exact match, the system prompts for a lower and upper bound (e.g., 3.50 to 3.75). It then traverses the AcademicInfo array, evaluating the logical condition (CGPA >= min && CGPA <= max). Every record satisfying this condition is dynamically mapped to its corresponding personal data using the relational index and displayed sequentially (e.g., fetching IDs 1008, 1003, and 1014).

3.2.11 Comprehensive Report Card Generation

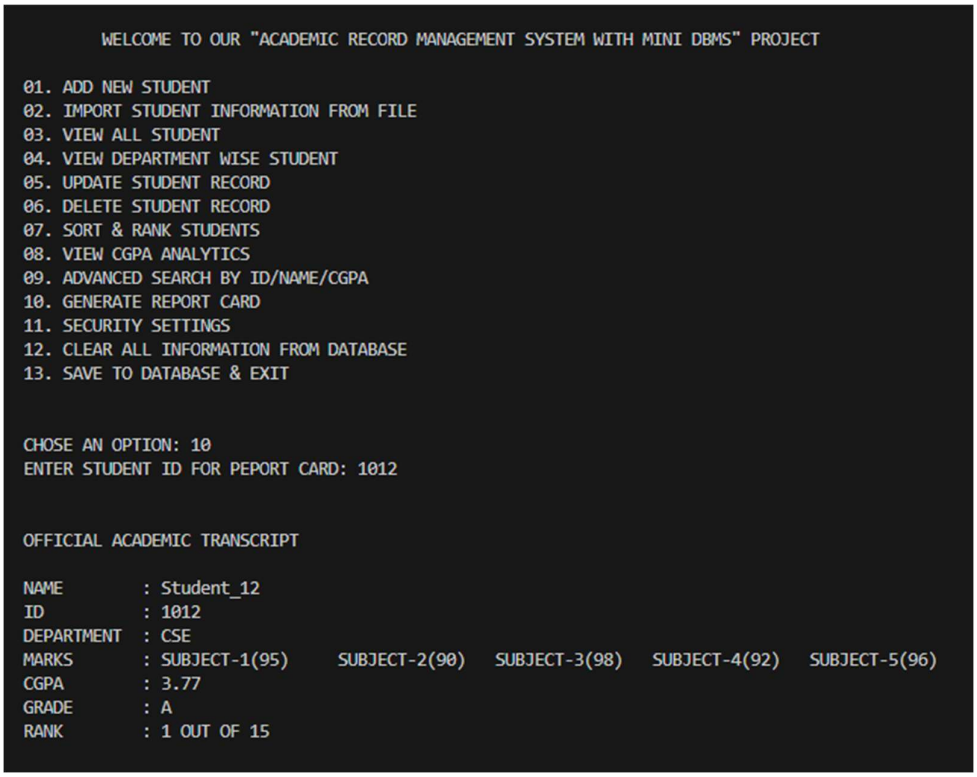


Fig 3.2.11 Report Card Generator

Discussion on Data Aggregation and Formatted Output: This output demonstrates the system's ability to aggregate relational data and present it through a highly formatted, user-friendly interface, mimicking an "Official Academic Transcript." The module is triggered by inputting a specific Student ID (e.g., 1012).

Upon receiving the query, the underlying algorithm performs a cross-structural data retrieval. It seamlessly fetches demographic details (Name, ID, Department) from the PersonalInfo array and maps them precisely with the corresponding academic metrics (Subject-wise marks, CGPA, and Letter Grade) from the AcademicInfo array.

A notable technical highlight of this module is the dynamic rank presentation (e.g., RANK: 1 OUT OF 15). The system not only retrieves the student's individual merit position but also cross-references the global counter variable to display the total number of active records in the database. This proves that the internal sorting mechanisms and database states are perfectly synchronized and capable of generating comprehensive, real-world reporting artifacts.

3.2.12 Security Management

Fig 3.2.12 Security Management

Discussion on Credential Management and Recovery Logic: These outputs demonstrate the system's robust credential management module, primarily handled through the changePass() function. Modifying the administrative access requires strict verification, which the system handles through a dual-layered authentication mechanism:

1. Standard Authentication: The system initially prompts for the current (old) password, masking the character inputs with asterisks using the getch() function for privacy. It then dynamically compares this input against the decrypted credentials stored in the password.bin file.
2. Secondary Recovery Mechanism: As demonstrated in the execution, if the administrator inputs an incorrect old password or forgets it, the system does not immediately terminate. Instead, it triggers a fallback recovery protocol. By successfully answering a predefined security question (e.g., "WHAT IS YOUR PET NAME?"), the authorized user can securely bypass the forgotten password barrier.

Once identity is verified via either of these methods, the system allows the user to input a new password and requires a confirmation entry. A conditional logic block checks if both the new password and the confirmation match exactly. Upon successful validation, the system encrypts the new password string (utilizing XOR encryption logic) and overwrites the existing binary file. This guarantees that the updated credentials are persistently saved and instantly active for all subsequent login attempts.

3.2.13 Data Deletion & Memory Deallocation Module

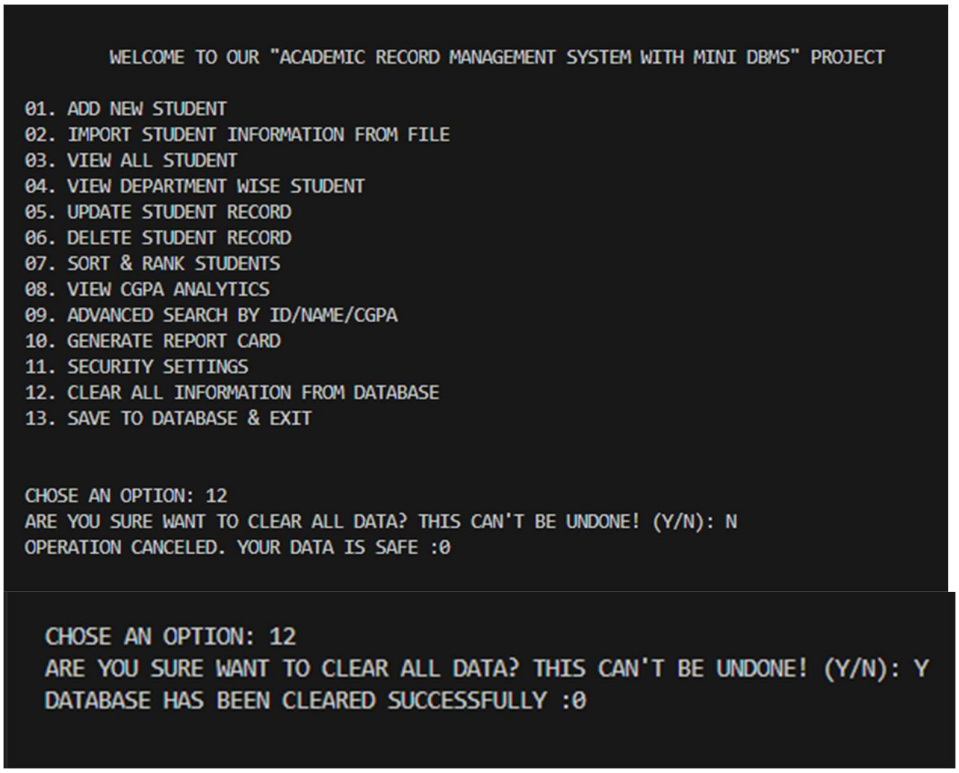


Fig 3.2.13 Data Deletion & Memory Deallocation Module

Discussion on Deletion Logic and Data Integrity: These outputs illustrate the system's data removal capabilities, a critical component of standard CRUD (Create, Read, Update, Delete) operations. Depending on the user's action, this module handles the safe eradication of records from both the volatile memory and the persistent storage.

1. In-Memory Deletion: When a specific record is targeted for deletion, the system first executes a linear search to find its exact index. Once located, the algorithm removes the data by systematically shifting all subsequent elements in the dynamically allocated arrays (PersonalInfo and AcademicInfo) one position to the left. This effectively overwrites the target record and closes the gap, keeping the array contiguous. The global student counter variable is then decremented by one.
2. Memory Optimization: If the module is executing a total database clear, it utilizes the free() function to safely deallocate the dynamically assigned memory blocks, preventing potential memory leaks and resetting the system state.
3. Persistent Storage Synchronization: Deleting data only from the RAM is insufficient. To ensure the deletion is permanent, the system automatically opens the local storage files (db_personal.txt and db_academic.txt) in write mode ("w"). This action truncates the files (wiping the old data) and rewrites the updated, shifted array back into the files. Consequently, the deleted record is permanently removed from the system.

This module ensures that the software can efficiently discard obsolete data without corrupting the relational mapping of the remaining active records.

3.2.14 Save Database and System Termination

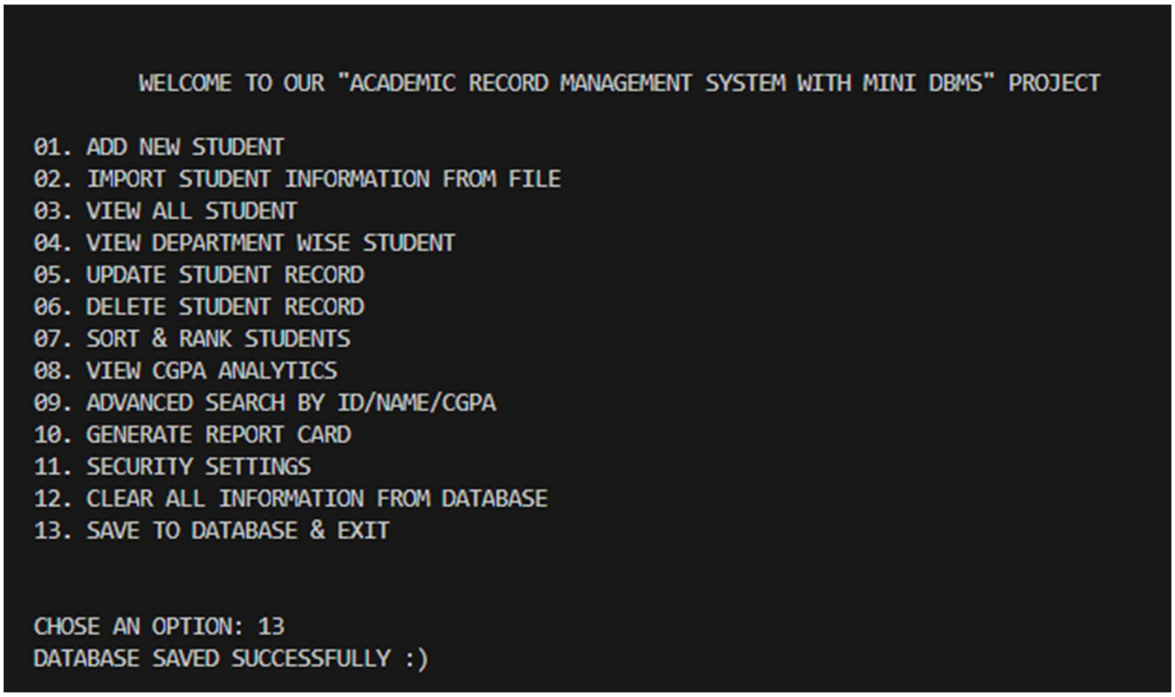


Fig 3.2.14 Save Database and System Termination

Discussion on Permanent Storage and Commit Logic: This output represents the final and most critical phase of the database lifecycle within this application: Permanent Data Persistence. While the system processes all student records in high-speed volatile memory (RAM) during the session, it requires a "Commit" operation to ensure these changes are not lost upon termination.

Option 13 serves as this final commit trigger. When selected, the program executes a specialized file-handling sequence where it opens the external database files (db_personal.txt and db_academic.txt) in write mode and performs a complete data dump from the dynamic memory arrays to the physical storage. This ensures that every addition, update, or deletion performed during the current runtime is permanently archived.

If the application is closed abruptly without utilizing this option, the session data remains only in the RAM and is subsequently cleared, resulting in total data loss. Therefore, this module is essential for bridging the gap between temporary in-memory processing and long-term storage, ensuring that the system maintains data integrity and continuity across multiple sessions.

Chapter 4

Engineering Standards and Mapping

This chapter explores the engineering standards applied in developing the Academic Record Management System. It also evaluates the project's broader impacts on society, environment, ethics, and its long-term sustainability.

4.1 Impact on Society, Environment and Sustainability

The implementation of this academic management software significantly modernizes educational administration. By transitioning from paper-based to digital records, it positively influences societal efficiency, reduces environmental footprints, and promotes sustainable data practices.

4.1.1 Impact on Life

This system drastically improves the daily lives of educational administrators, teachers, and students. By automating tedious tasks like grading, ranking, and record-keeping, it significantly reduces manual workload and minimizes human errors. Administrators can retrieve student information instantly, saving valuable time and reducing workplace stress. Ultimately, it ensures a smoother academic experience, allowing educators to focus more on teaching rather than getting bogged down by administrative paperwork.

4.1.2 Impact on Society & Environment

Societally, this project promotes digital literacy and modernizes institutional operations, reflecting a progressive educational ecosystem. Environmentally, the system champions a paperless approach by replacing physical logbooks and printed report cards with digital database files. This reduction in paper consumption directly contributes to decreasing deforestation and lowering the carbon footprint associated with manufacturing and transporting traditional office supplies, fostering a greener and more eco-friendly academic environment.

4.1.3 Ethical Aspects

Ethical considerations are central to this system, particularly concerning data privacy and security. The integration of an admin login and XOR password encryption ensures that sensitive student information, such as academic grades and personal details, remains protected from unauthorized access. The system is designed to treat all student data impartially, ensuring fairness in automated grading and ranking without any systemic bias, thereby maintaining high ethical standards.

4.1.4 Sustainability Plan

The software is built with long-term sustainability in mind, utilizing dynamic memory allocation to efficiently handle growing datasets without crashing. Future sustainability efforts could involve migrating the current file-based storage to a robust relational database (like MySQL) and upgrading the console interface to a graphical user interface (GUI). Regular code maintenance and security updates will ensure the system remains reliable and adaptable for years to come.

4.2 Project Management and Team Work

This project was executed as a group effort, where tasks were divided according to the individual strengths and capabilities of each team member. While the core programming was handled by a single developer, the other members contributed significantly through brainstorming, testing, and documentation to ensure the successful completion of the project.

4.2.1 Team Member Contributions

- Md Ashikur Alom Sarkar (ID: 253-15-204) - Lead Developer: Solely responsible for the system architecture and writing the entire C code. Implemented all the advanced features including dynamic memory allocation, Relational Database mapping (Mini DBMS), File I/O operations, XOR password encryption, and the recursive QuickSort algorithm.

- Athoi Mondal Katha (ID: 253-15-439) - Conceptualization & Documentation: Contributed by brainstorming ideas for the user interface and the menu options. Assisted in visualizing how the user would interact with the system. Was primarily responsible for typing, formatting, and organizing the final engineering project report, as well as performing basic user-level testing.

- Aditya Nag Saptam (ID: 253-15-558) - Data Collection & Demo Preparation: Helped by researching the standard CGPA grading policy to implement in the system and collected dummy data (names, IDs, marks) to test the database. Also took the responsibility of preparing the test cases and the step-by-step execution plan for the live project demonstration in front of the course instructor.

4.2.2 Cost Analysis and Budget Required

Since this is a console-based software developed using open-source tools (GCC Compiler, VS Code), the direct software licensing cost is zero. However, a hypothetical commercial budget for developing and deploying this system in a local institution includes:

- Development Cost (Hypothetical Developer Hours): 15,000 BDT
- Hardware Setup (Local Desktop Server for Admin): 40,000 BDT
- Initial Training & Deployment: 5,000 BDT
- Total Primary Budget: 60,000 BDT

Alternate Budget and Rationales: An alternate budget would involve migrating the current local .txt file-based storage to a professional Cloud Relational Database (e.g., AWS RDS or Google Cloud SQL) for remote, multi-user support.

- Cloud Hosting & DB License: ~1,000 BDT / month (12,000 BDT / year)
- *Rationale:* While the primary budget relies on cost-free local storage (suitable for a single computer), the alternate budget justifies the recurring cloud costs by providing enhanced data security, remote accessibility, and protection against hard drive failures.

4.2.3 Revenue Model

If commercialized, the software will follow a B2B (Business-to-Business) One-Time Licensing Model.

- License Fee: Educational institutions (schools/colleges) can purchase the standalone software for a fixed fee of 15,000 BDT per campus.
- Maintenance Subscription: An optional 3,000 BDT/year subscription for technical support, database cleanup, and periodic security updates. This ensures a steady stream of secondary revenue while keeping the initial cost highly affordable for local institutions.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	CO1 covers PO1 by applying fundamental engineering knowledge and C programming concepts. In this project, basic problem-solving techniques were manipulated using loops, conditional logic, and custom functions (e.g., gradeCalculate, checkLogin) to build the foundational structure of the Academic Record Management System.
PO2	CO2 covers PO2 by demonstrating the ability to analyze complex system requirements and select appropriate data structures. To handle realistic database constraints, we analyzed the problem and implemented dynamic memory allocation (malloc, realloc), Arrays of Structures for the Mini DBMS, and the advanced recursive QuickSort algorithm for efficient student ranking.
PO9	CO3 covers PO9 as it maps to functioning effectively as an individual and as a member in diverse teams. To design and implement this software project, the workload was strategically divided among team members (Lead Developer, Documentation, and Demo Preparation), demonstrating coordination and integration of team efforts to achieve the project goals.
PO10	CO4 covers PO10 by emphasizing effective communication of complex engineering activities. This is demonstrated through the creation of a well-structured, user-friendly interactive console interface for the software, alongside the drafting of a comprehensive engineering report and preparing the live demonstration steps to present technical concepts to the course instructor.

4.3.2 Complex Problem Solving

- CO1 is linked to EP1 as developing the foundational structure of the Academic Record Management System requires applying in-depth engineering knowledge of basic C programming concepts, such as loops, conditional logic, and functions. EP2 is also relevant as students must balance different problem-solving techniques and manage conflicting requirements, such as ensuring a secure login system while maintaining an accessible user interface.

- CO2 is linked to EP1 because designing the Mini DBMS using dynamic memory allocation (malloc, realloc) and structures requires a deep, first-principles understanding of advanced programming concepts. EP2 is also relevant because building realistic features, like implementing the QuickSort algorithm and File I/O operations, involves balancing conflicting requirements, such as optimizing execution speed versus managing memory consumption effectively.

Table 4.2: Mapping with complex problem solving.

EP1 Depth of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓	✓					

4.3.3 Engineering Activities

In this section, a mapping with engineering activities is provided. The rationale for the specific mapped activity is detailed below.

Mapping with EA1 (Range of resources)

The development of the Academic Record Management System required the selection and application of a specific range of engineering resources. This included utilizing open-source software development tools (such as VS Code and the GCC Compiler) and leveraging standard C programming libraries (stdio.h, stdlib.h, string.h). Furthermore, building the Mini DBMS involved utilizing hardware memory resources efficiently through dynamic memory allocation (malloc, realloc) and managing persistent data storage using File I/O resources (db_personal.txt and db_academic.txt).

Table 4.3: Mapping with engineering activities

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity
✓				

Chapter 5

Conclusion

This chapter concludes the project by summarizing the key features and technical achievements of the Academic Record Management System. It also evaluates the current limitations of the software and outlines potential future enhancements.

5.1 Summary

The Academic Record Management System was successfully developed as a comprehensive console-based application using the C programming language. It effectively demonstrates foundational database management principles by separating data into relational structures (Personal and Academic information) and linking them via unique IDs. The project successfully integrated dynamic memory allocation (malloc and realloc), ensuring efficient resource management without rigid array limits. Furthermore, advanced features such as XOR password encryption, automated CGPA grading, recursive QuickSort for academic ranking, and robust File I/O for persistent data storage were seamlessly implemented. Overall, this project serves as a highly practical implementation of data structures, algorithms, and secure software development practices.

5.2 Limitation

1. Lack of Graphical Interface: The system currently relies entirely on a command-line console interface (CLI) and lacks a modern Graphical User Interface (GUI) for better user experience.
2. File-Based Storage Constraint: Data is stored in standard text files (.txt), which is less secure and significantly slower for querying massive datasets compared to actual SQL databases.
3. Sequential Searching: The program utilizes linear search for finding student records, which may become inefficient and time-consuming if the database grows to thousands of entries.
4. Single-User Environment: The software functions solely as a standalone desktop application and lacks multi-user or network support for simultaneous access.
5. Basic Input Validation: While functional, the system's input validation is somewhat rigid; accidentally entering characters instead of numbers in certain fields might cause terminal errors.

5.3 Future Work

1. GUI Implementation: Upgrading the user experience by integrating graphical libraries (such as GTK or Qt) or transitioning the logic into a web-based architecture.
2. Database Migration: Replacing the traditional file handling system with a proper relational database engine like SQLite or MySQL for enhanced security, integrity, and scalability.
3. Algorithm Optimization: Implementing advanced data structures like Hash Tables or Binary Search Trees (BST) to make data retrieval and searching operations instantaneous.
4. Role-Based Access Control: Introducing a multi-tier login system with distinct privileges for different users, such as "Admin," "Teacher," and "Student" modules.
5. Advanced Data Analytics: Expanding the analytics dashboard to include graphical charts (like bar graphs for department-wise performance) and automated warning systems for underperforming students.

References

[1] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.